



案例特训专题

【软件工程篇】



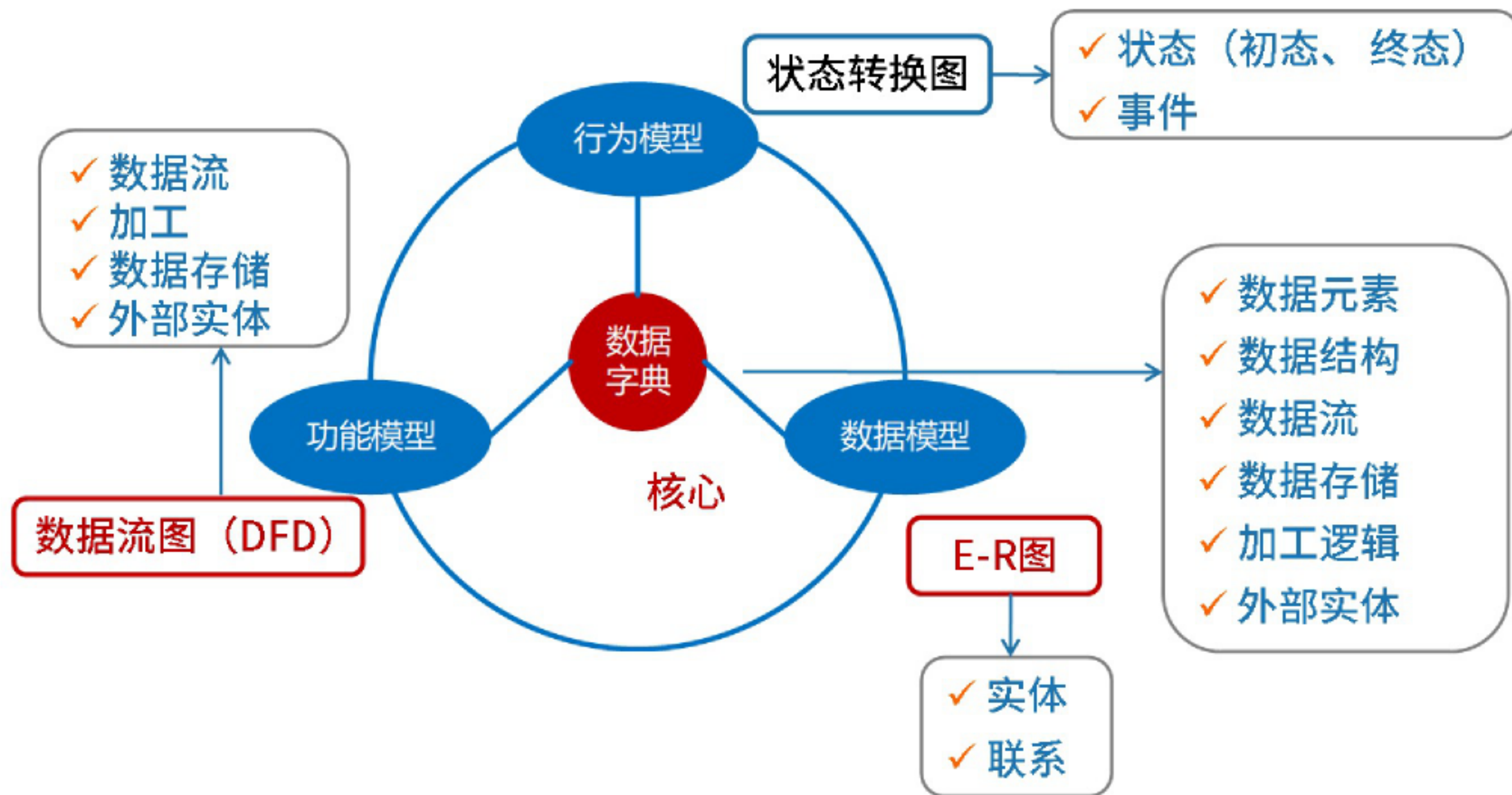
课程内容提要



- 需求分析 (★★★★)
- 面向对象设计 (★★)



结构化需求分析 (SA)





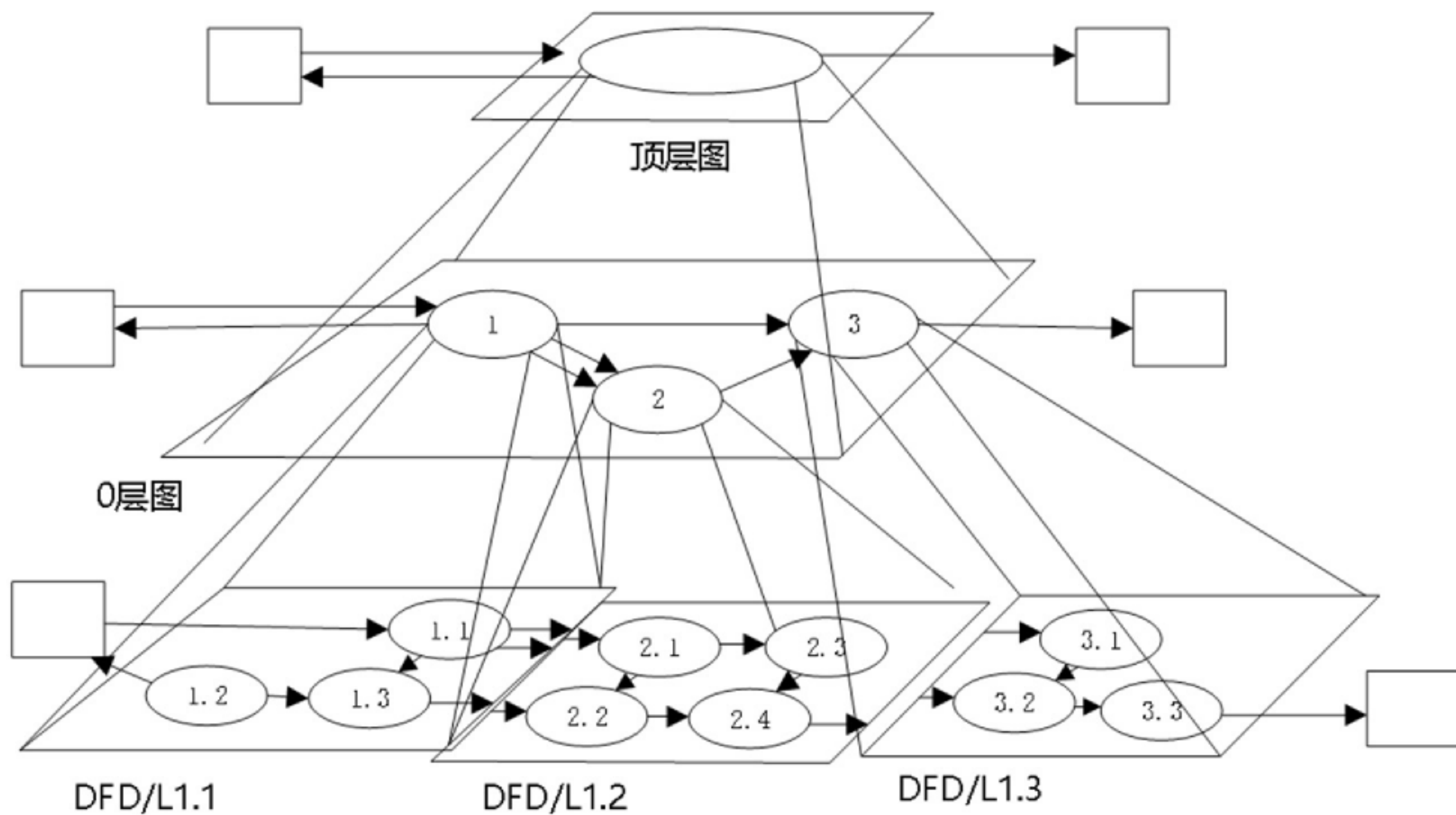
数据流图（DFD）



元 素	说 明	图 元
数据流	由一组固定成分的数据组成，表示数据的流向。每个数据流通常有一个合适的名词，反映数据流的含义	
加工	加工描述了输入数据流到输出数据流之间的变换，也就是输入数据流做了什么处理后变成了输出数据流	
数据存储 (文件)	用来表示暂时存储的数据，每个文件都有名字。流向文件的数据流表示写文件，流出的表示读文件	
外部实体	指存在于软件系统外的人员或组织	



数据流图 (DFD)





数据流图 (DFD)

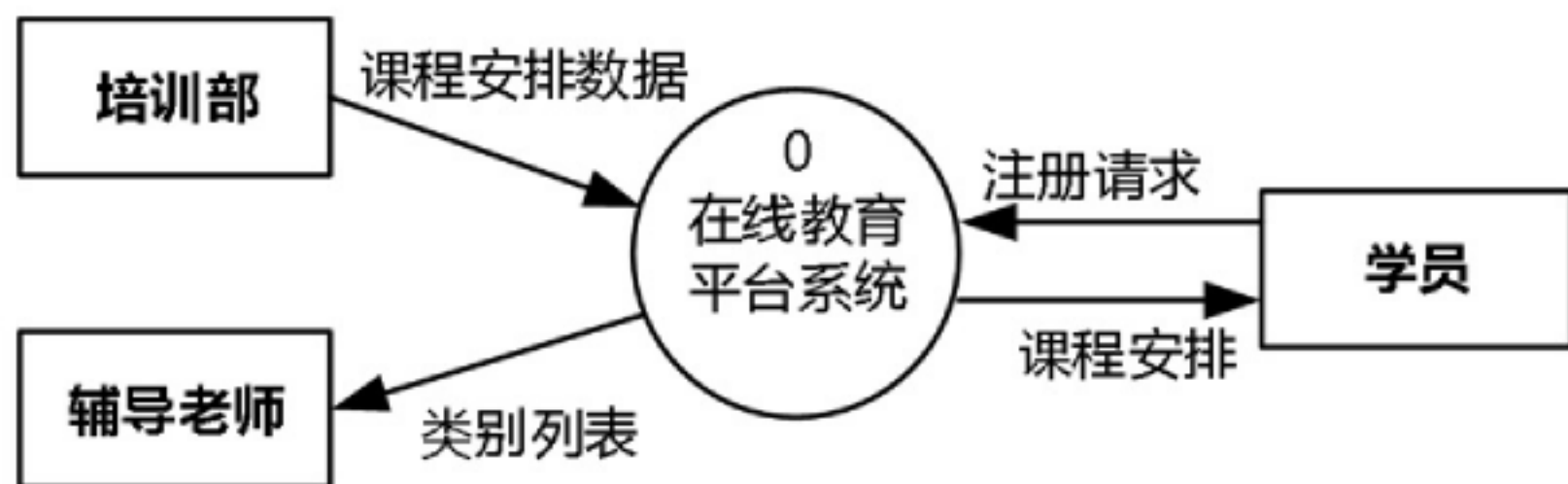


图1 顶层图

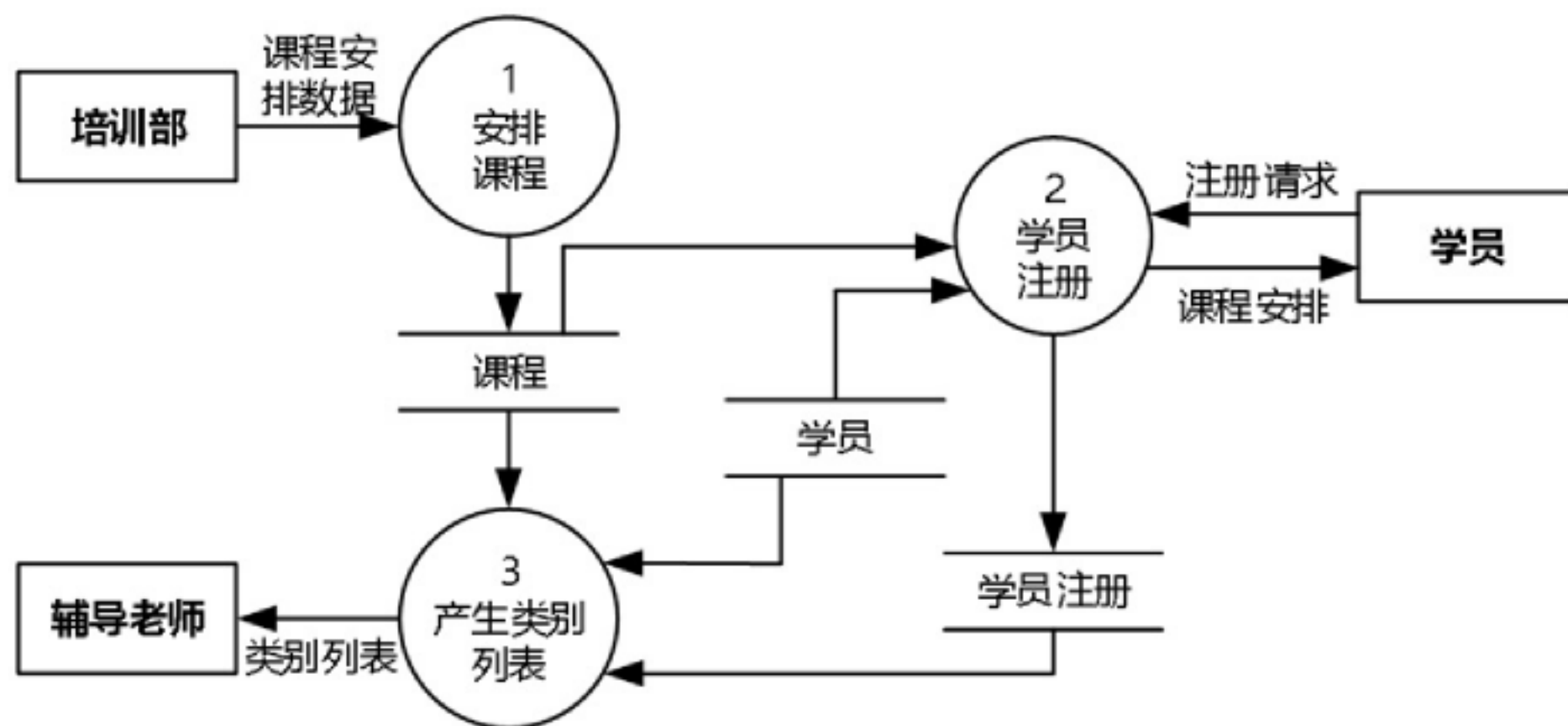


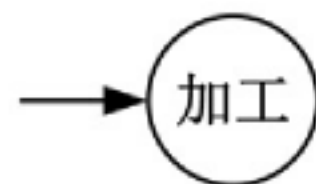
图2 0层图



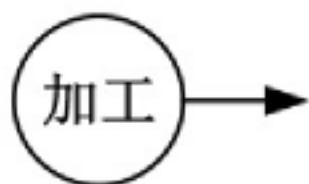
数据流图平衡原则



- 父图与子图之间的平衡
- 子图内平衡



(a) 黑洞



(b) 奇迹



(c) 灰洞

异常现象

【黑洞】 一个加工只有输入数据流而无输出数据流。

【奇迹】 一个加工只有输出数据流而无输入数据流。

【灰洞】 若一个加工的输入数据流无法通过加工产生输出流。



数据流图平衡原则



图2 0层数据流图↓

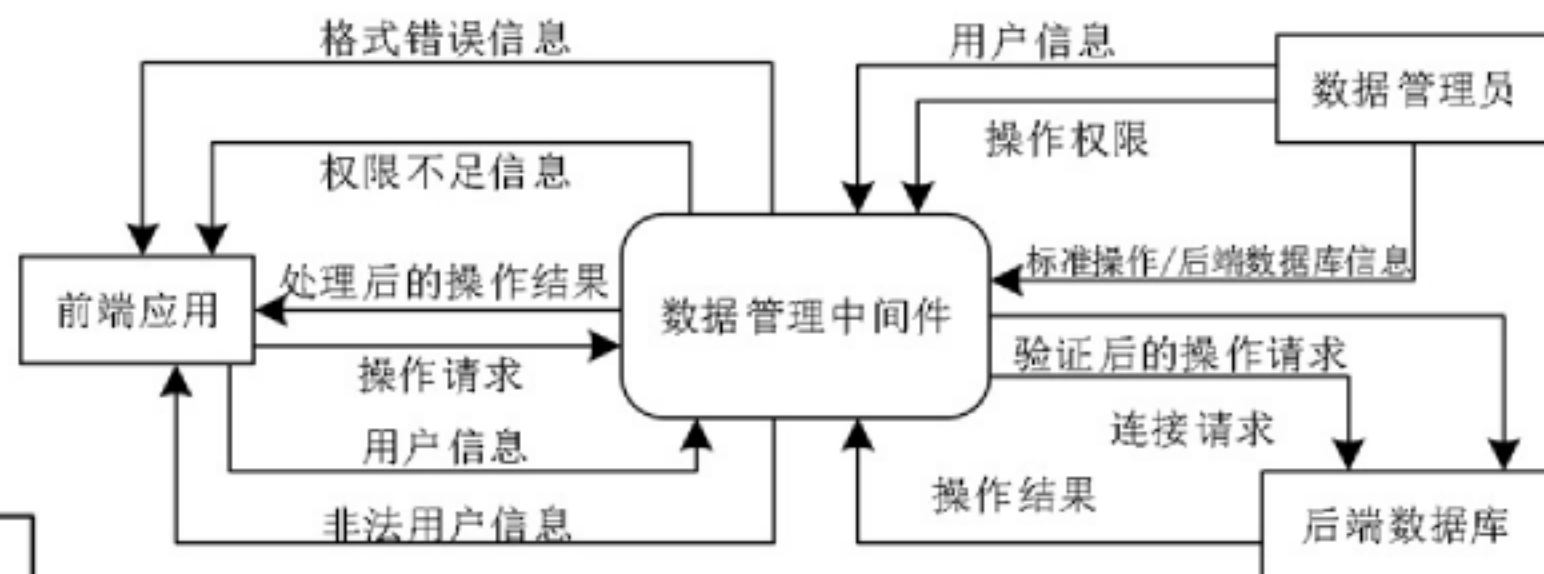
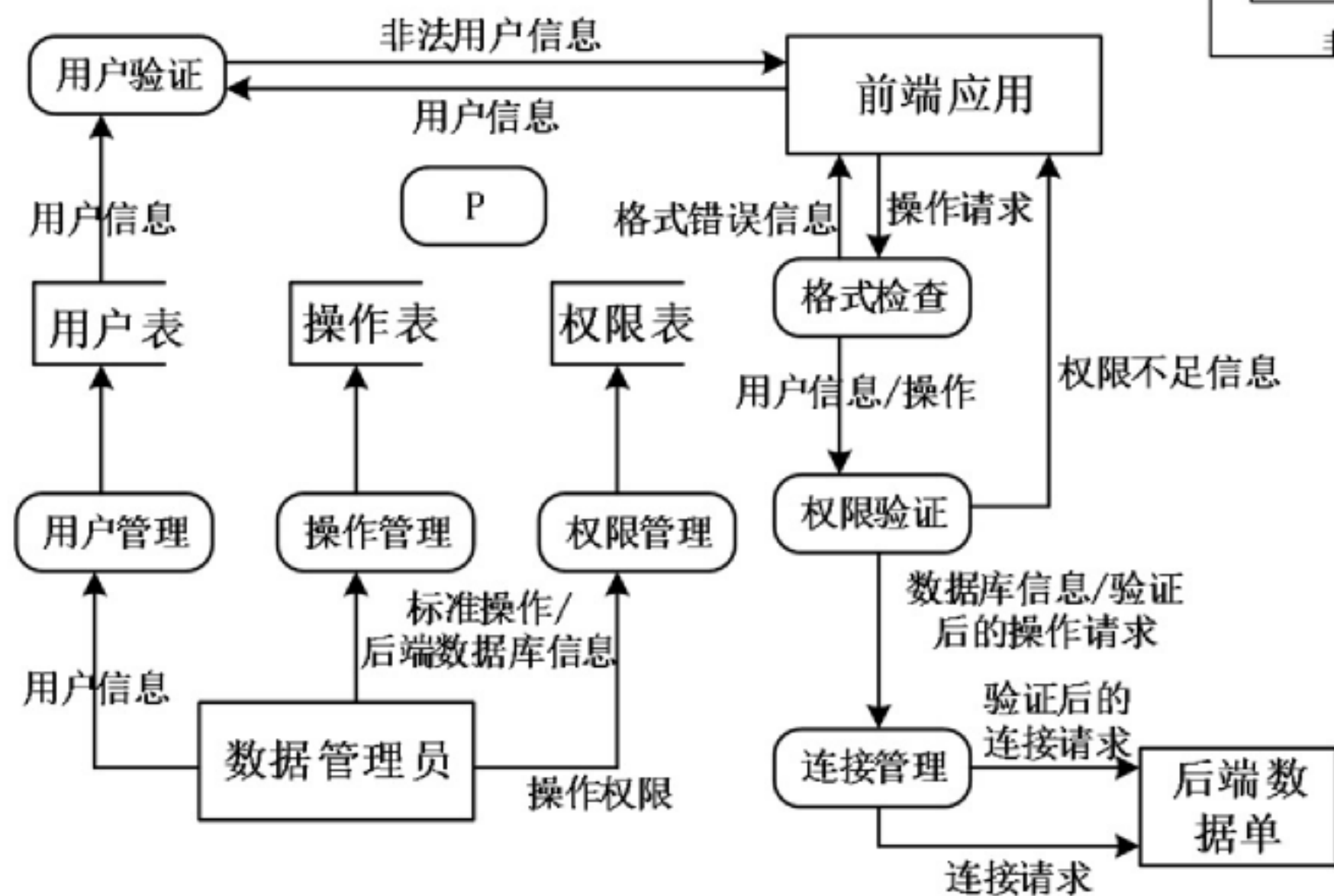


图1 顶层数据流图↑



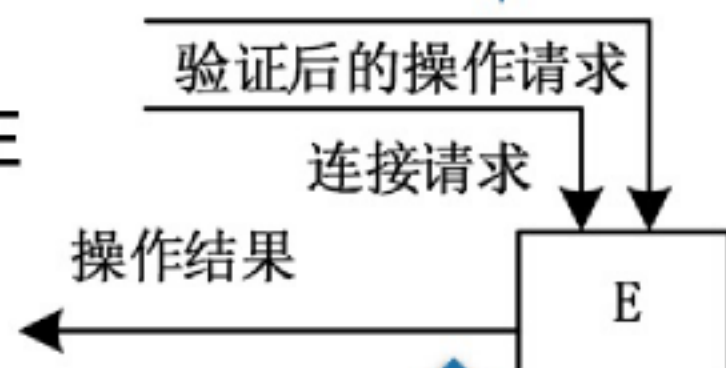
答题技巧



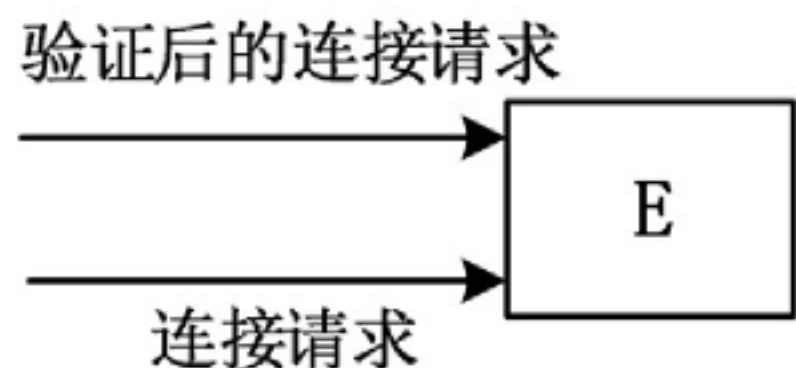
- 详细分析试题说明
- 利用数据平衡原则

数据管理员可通过中间件进行用户管理、操作管理和权限管理。用户管理维护用户信息，用户信息（用户名、密码）存储在用户表中；操作管理维护数据实体的标准操作及其所属的后端数据库信息，标准操作和后端数据库信息存放在操作表中；权限管理维护权限表，该表存储用户可执行的操作信息。

顶层图中的实体E



0层图中的实体E



- ✓ 数据管理员是一个外部实体；
- ✓ 中间件中有“用户管理”“操作管理”“权限管理”这些加工；
- ✓ 中间件中有“用户表”这个数据存储，且该存储与“用户管理”相关；
- ✓ 后端数据库是一个外部实体；
- ✓ 中间件中有“操作表”这个数据存储，且该存储与“操作管理”相关；
- ✓ 中间件中有“权限表”这个数据存储，且该存储与“权限管理”相关。



答题技巧



一、补充实体

实体可能是：

- (1) 人物角色：如客户、管理员、主管、经理、老师、学生
- (2) 组织机构：如银行、供应商、募捐机构
- (3) 外部系统：如银行系统、工资系统、后台数据库（当要开发的是中间件时）

二、补充存储

存储的文字方面特征：“**文件” “**表” “**库” “**清单” “**档案”



答题技巧



三、补充数据流

1、数据平衡原则

- (1) 顶层图与0层图对比，是否有顶层图有，但0层图无的数据流，或反之。
- (2) 检查图中每个加工，是否存在只有入没有出，或只有出没有入，或根据输入的数据无法产生对应的输出的情况。

2、按题目说明与图进行匹配

说明中的每一句话，都能与图中有对应关系，当把说明中的实体与数据流标识出来之后，容易缩小对应范围，找出纰漏。

四、补充加工名

加工是用于处理数据流的，所以要补充加工名，可以把该加工涉及到的数据流，在说明中标识出来，再在数据流名称所在的句子中，找“动词+名词”的结构，分析是否可作为加工。

“动词+名词”如：生成报告、发出通知、批改作业、记录分数，当然这只是普遍情况，也有例外，如物流跟踪、用户管理。



UML图



UML图

静态图（结构图）

动态图（行为图）

- **类图**
- 对象图
- 构件图
- 部署图
- 制品图
- 包图
- 组合结构图

- **用例图**
- 顺序图
- 通信图
- 状态图
- 活动图
- 定时图
- 交互概览图

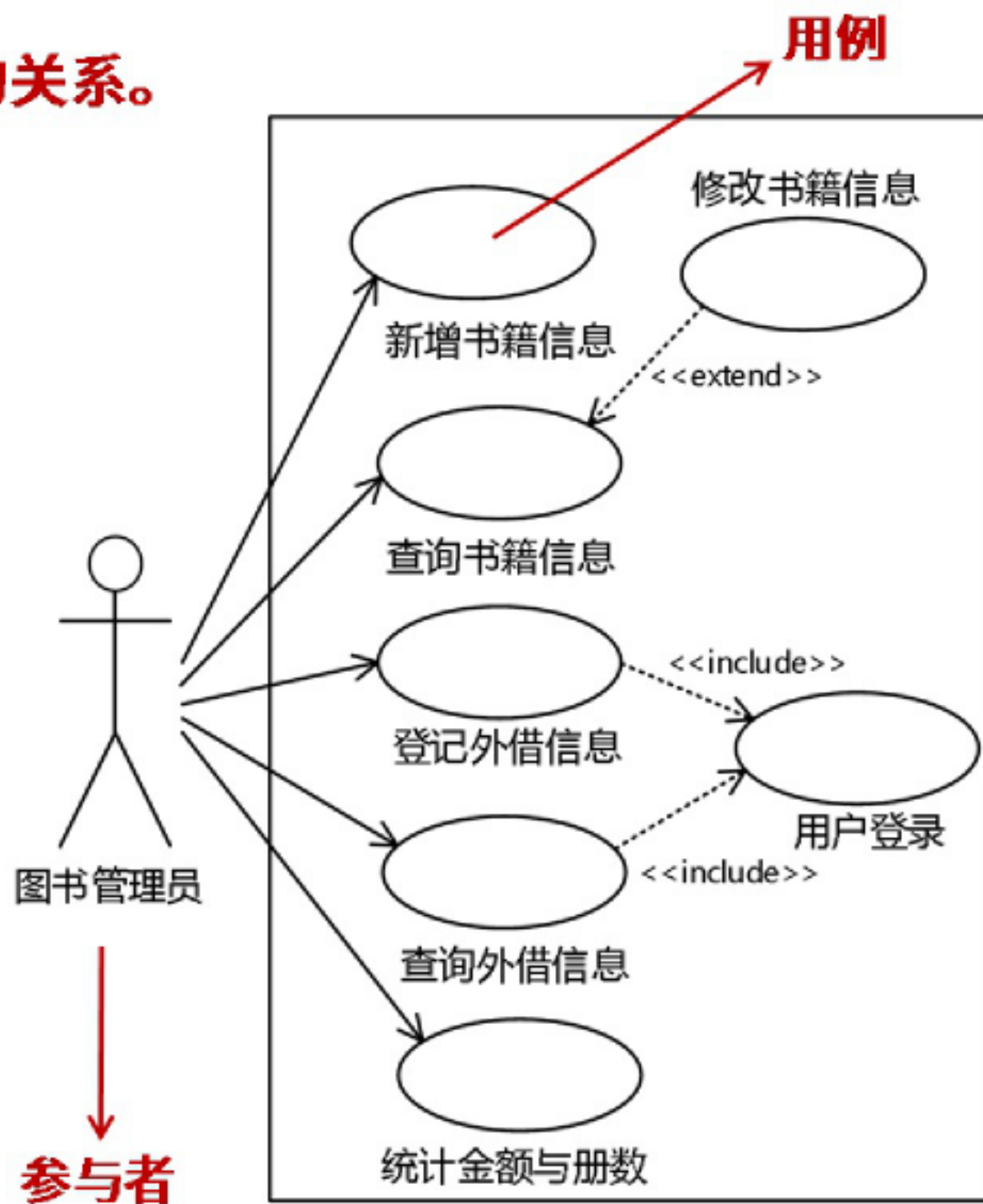


用例图



用例图描述一组用例、参与者及它们之间的关系。

- ✓ 用户角度描述系统功能；
- ✓ 参与者是外部触发因素；
- ✓ 用例是功能单元。





用例图



【用例模型】建立流程

第一步：识别参与者【参与者：用户、组织、外部系统，时间】

第二步：合并需求获得用例

第三步：细化用例描述

第四步：调整用例模型（可选步骤）【关系包括：包含关系、扩展关系、泛化关系】



用例图



细化用例描述【登录用例】

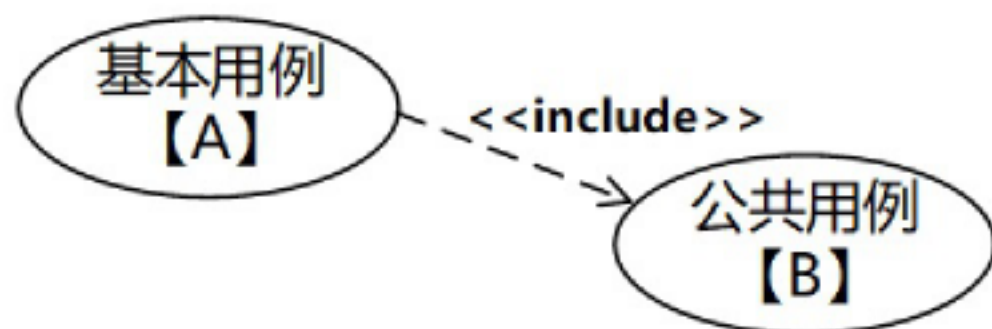
用例名称:	登录	
用例ID:	ESHOP_002.1	
角色:	普通用户	
用例说明:	用例主要功能是实现登录，起始于普通用户的登录	
前置条件:	启动程序，进入登录界面	
基本事件流:	参与者动作	系统响应
	1.用户输入基本信息（登录名和密码），点击确定按钮	2.系统查找数据库，看该用户是否在数据库中。若存在则进入主页面；若不存在，则进入2.2.2；若未输入，则进入2.2.2
其它事件流:	无	
异常事件流:	参与者动作	系统响应
	2.2.2未输入用户名 2.2.1用户名不存在 2.1.2未输入密码 2.2.2密码不正确	2.1.1提示用户名或密码不能为空 2.2.2提示用户名或密码不正确
后置条件	登录成功	



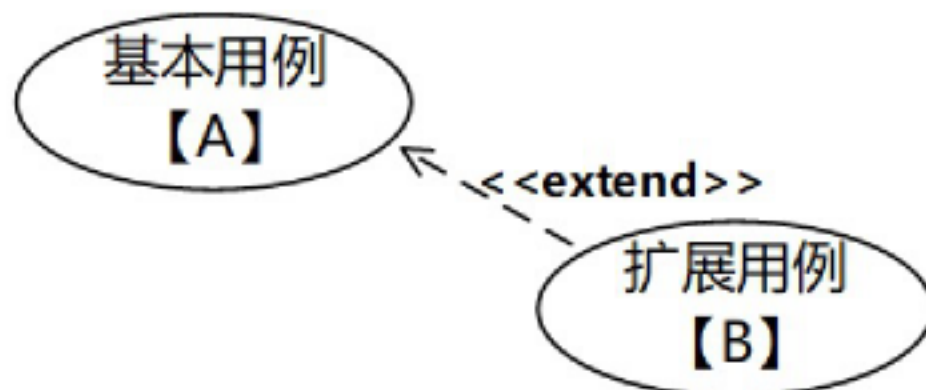
用例关系【包含、扩展、泛化】



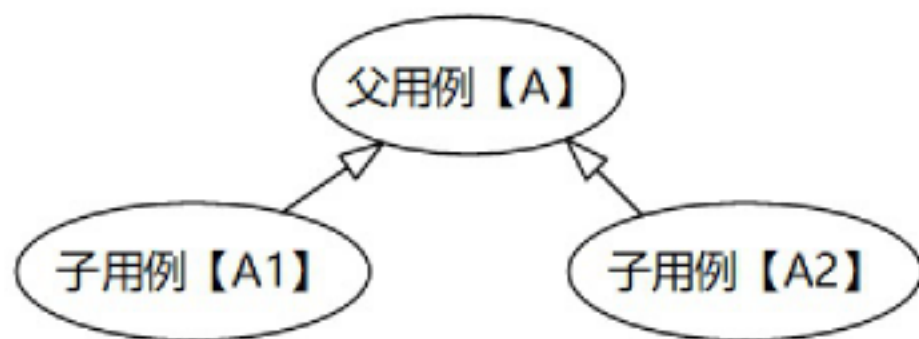
包含关系【使用关系】：从多个用例中提取公共行为，提取出来的公共用例称为抽象用例，而把原始用例称为基本用例。



扩展关系：一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个**基本用例**和一个或多个**扩展用例**。



泛化关系：当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。子用例继承了父用例所有的结构、行为和关系。

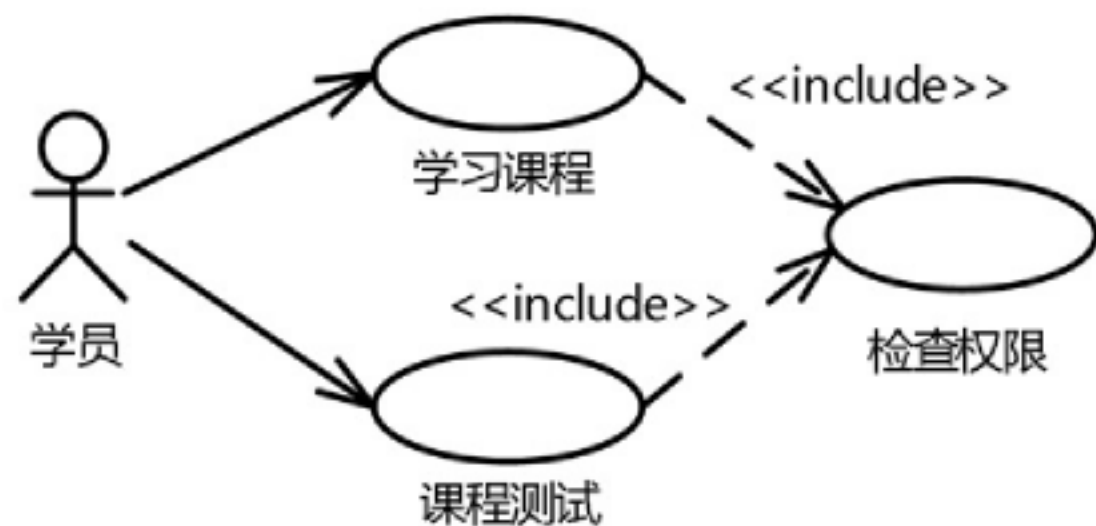




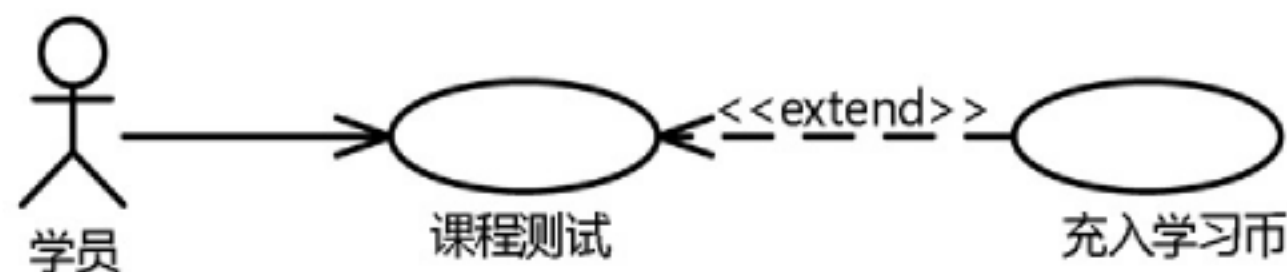
用例关系【包含、扩展、泛化】



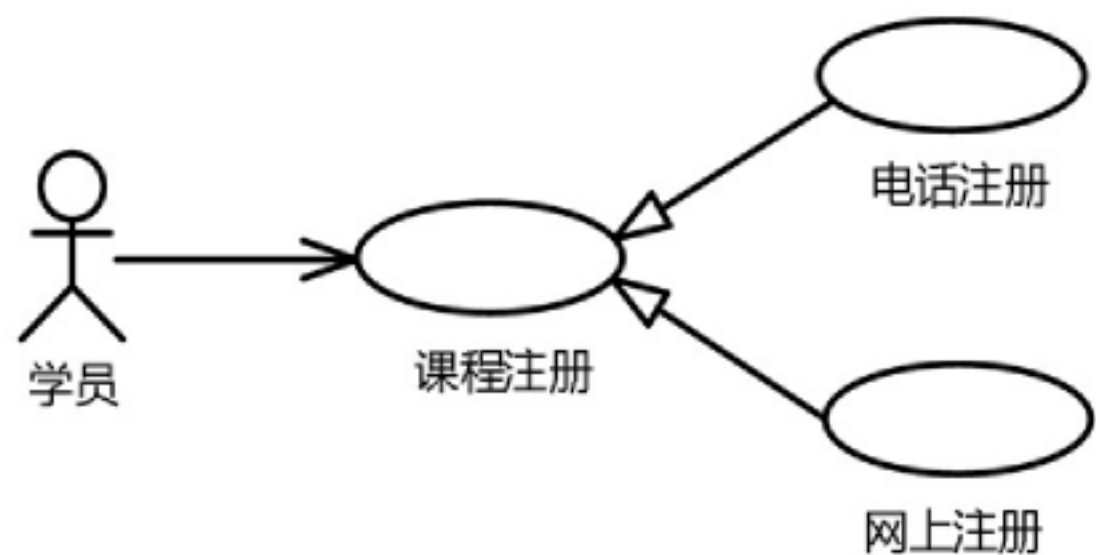
包含关系



扩展关系



泛化关系





习题讲解



在某银行业务的用例模型中：“取款”和“存款”两个用例中都需要执行查询余额的功能，将查询余额提取成独立的用例，那么“取款”和“存款”用例与“查询余额”用例之间的关系属于()。

- A. 扩展关系 B. 包含关系 C. 依赖关系 D. 继承关系

采用 UML 分析用户需求时，用例 UC1 可以出现在用例 UC2 出现的任何位置，那么 UC1 和 UC2 之间的关系是()。

- A. include B. extend C. generalize D. call



类图与对象图



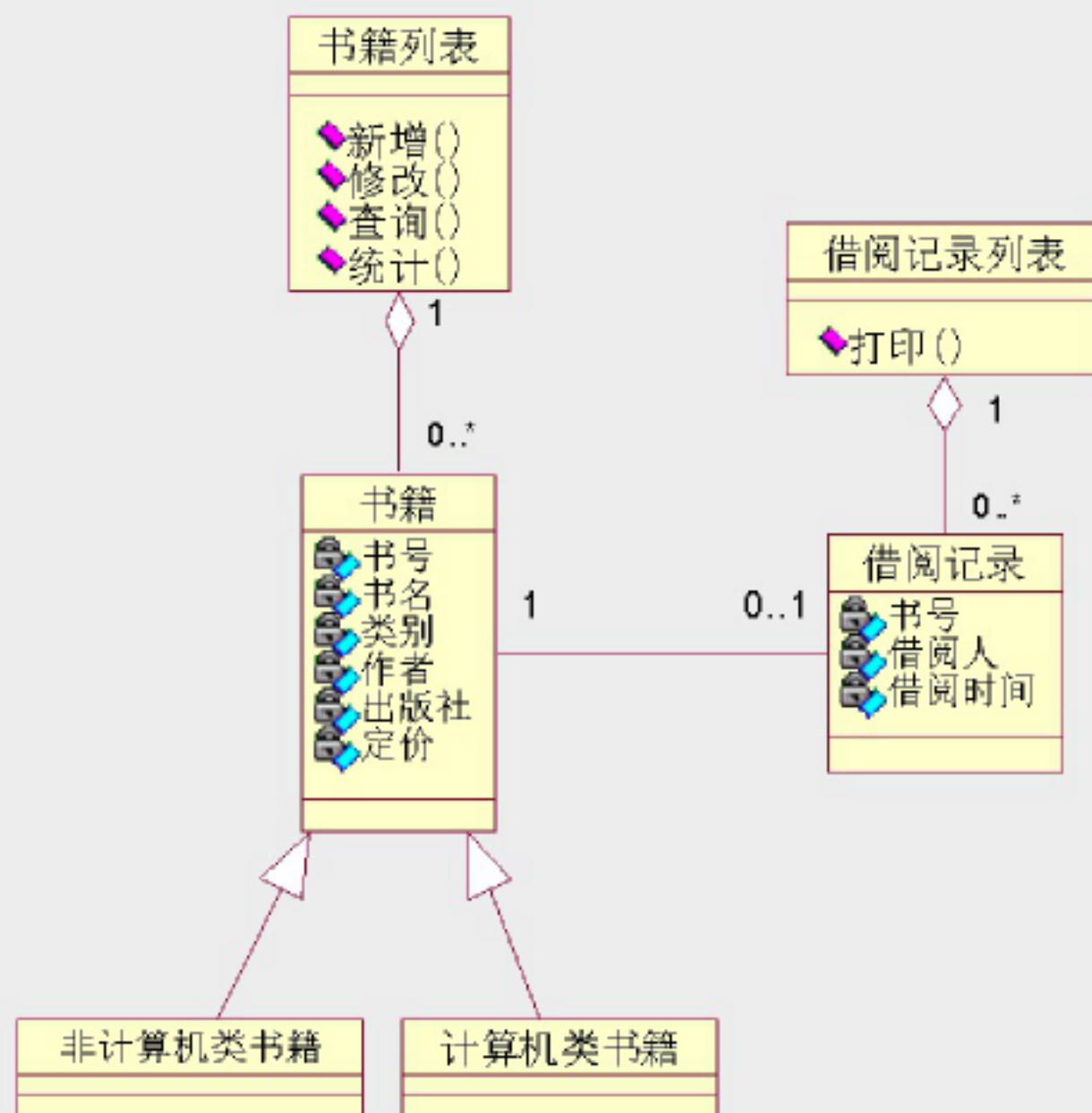
类图 (class diagram)：类图描述一组类、接口、协作和它们之间的关系。

对象图 (object diagram)：对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。

◆ 类名，方法名，属性名

◆ 多重度

◆ 关系





类图与对象图



1: 表示一个集合中的一个对象对应另一个集合中一个对象。

0..*: 表示一个集合中的一个对象对应另一个集合中的0个或多个对象。

(可以不对应)

1..*: 表示一个集合中的一个对象对应另一个集合中的一个或多个对象。

(至少对应一个)

*****: 表示一个集合中的一个对象对应另一个集合中的多个的对象。



类图与对象图



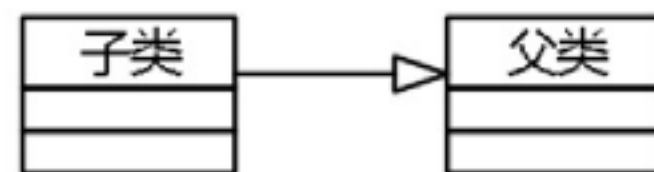
- ✓ **依赖关系**：一个事物发生变化影响另一个事物。
- ✓ **泛化关系**：特殊/一般关系。
- ✓ **关联关系**：描述了一组链，链是对象之间的连接。
 - **聚合关系**：整体与部分生命周期**不同**。
 - **组合关系**：整体与部分生命周期**相同**。
- ✓ **实现关系**：接口与类之间的关系。



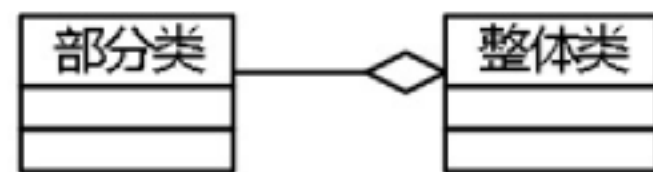
(a) 关联关系



(b) 依赖关系



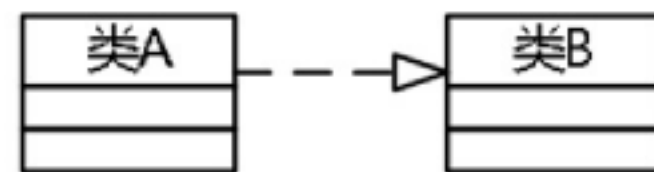
(c) 泛化关系



(d) 聚合关系



(e) 组合关系



(f) 实现关系



习题讲解



UML 用关系把事物结合在一起，（ ）描述一个事物发生变化会影响另一个事物的语义；（ ）描述特殊元素的对象可替换一般元素的对象。

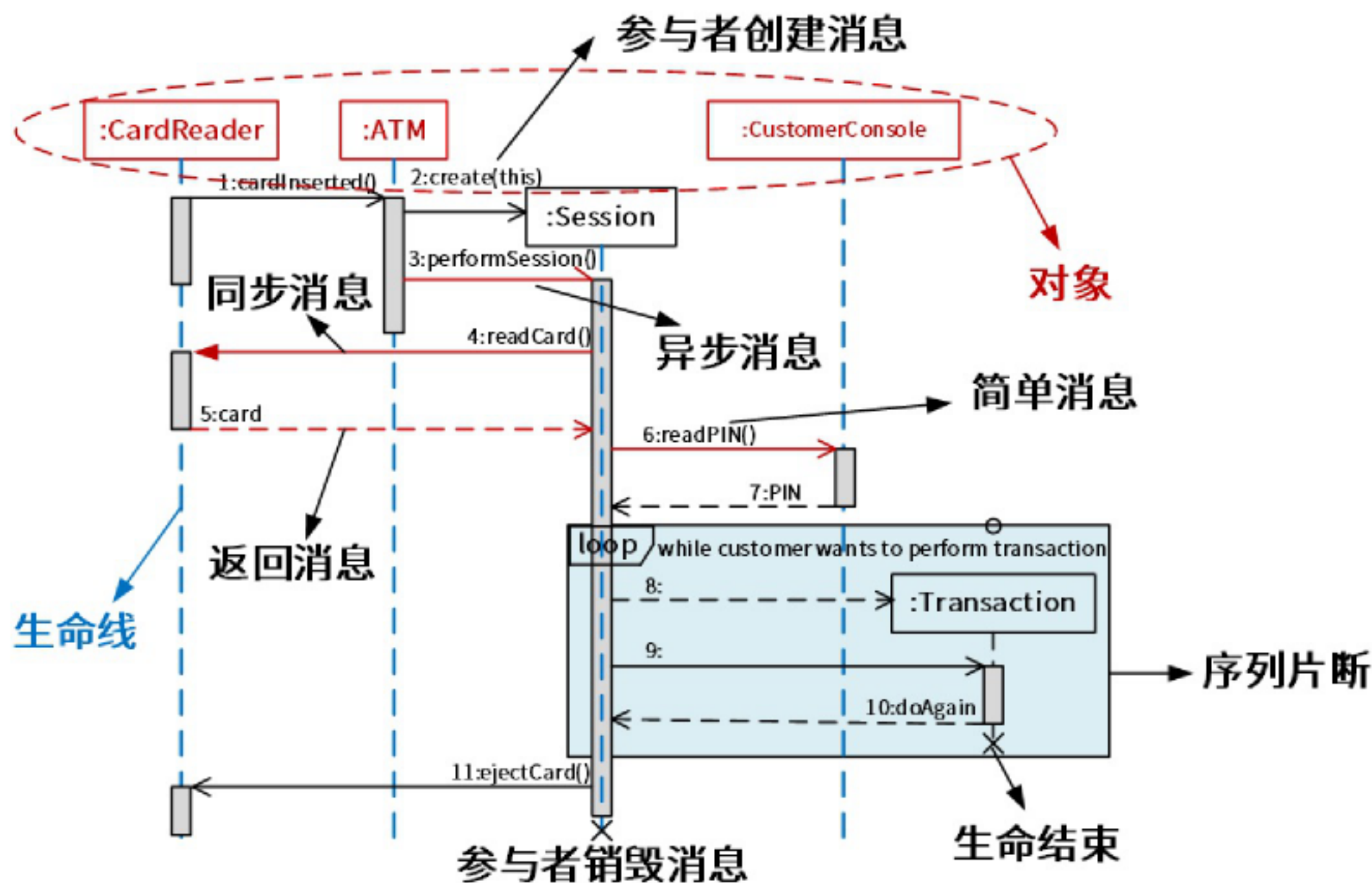
- | | | | |
|---------|---------|---------|---------|
| A. 聚合关系 | B. 关联关系 | C. 包含关系 | D. 依赖关系 |
| A. 实现关系 | B. 聚合关系 | C. 泛化关系 | D. 关联关系 |



顺序图



顺序图（sequence diagram，序列图）。顺序图是一种交互图（interaction diagram），它强调对象之间消息发送的顺序，同时显示对象之间的交互。

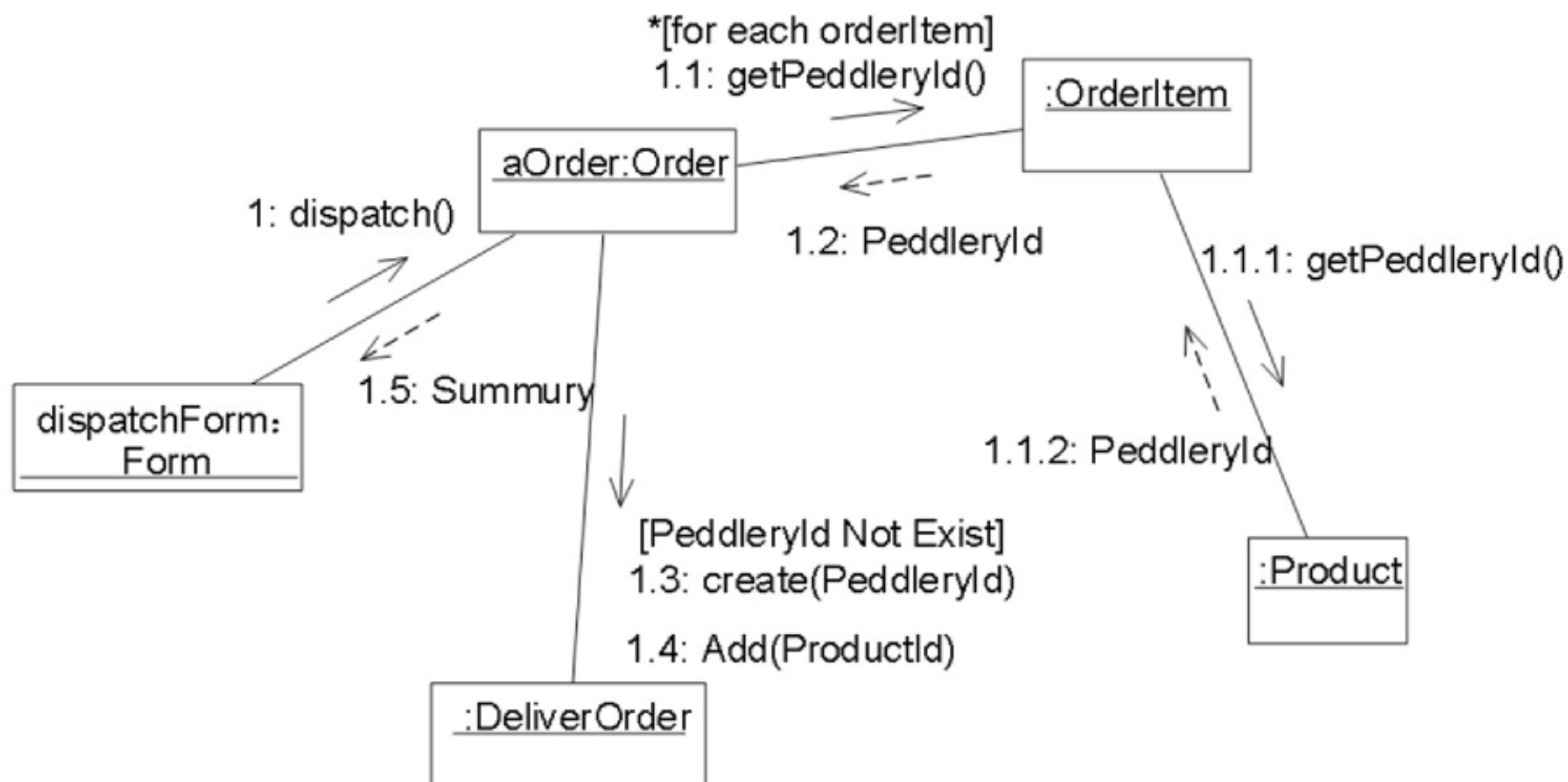




通信图（协作图）



通信图（communication diagram）。通信图也是一种交互图，它强调对象之间存在的消息收发关系，而不专门突出这些消息发送的时间顺序。

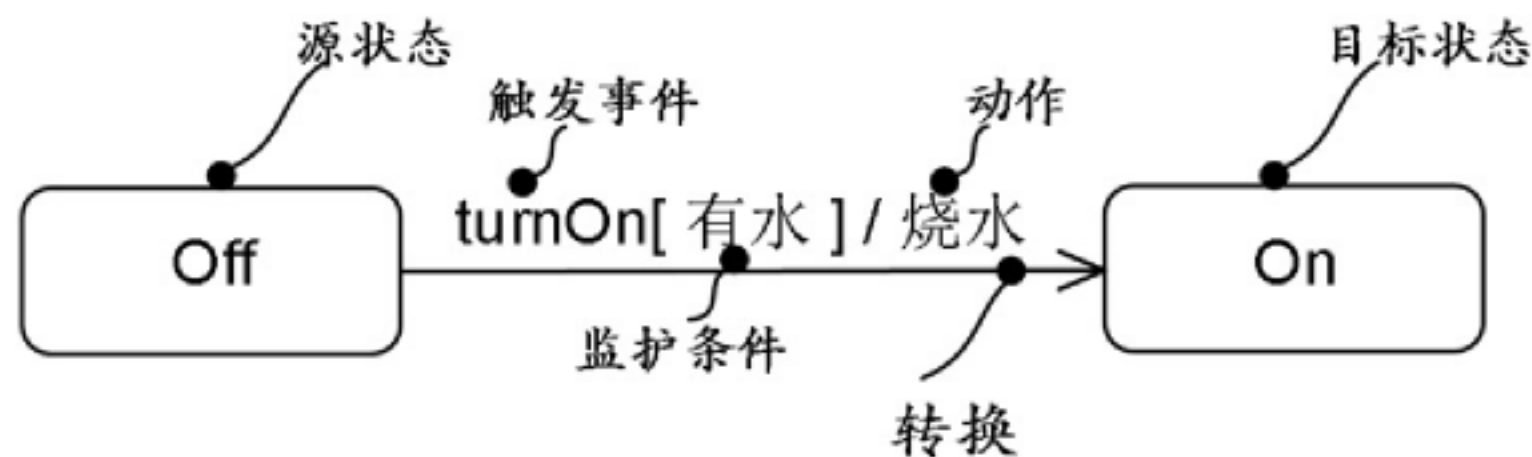
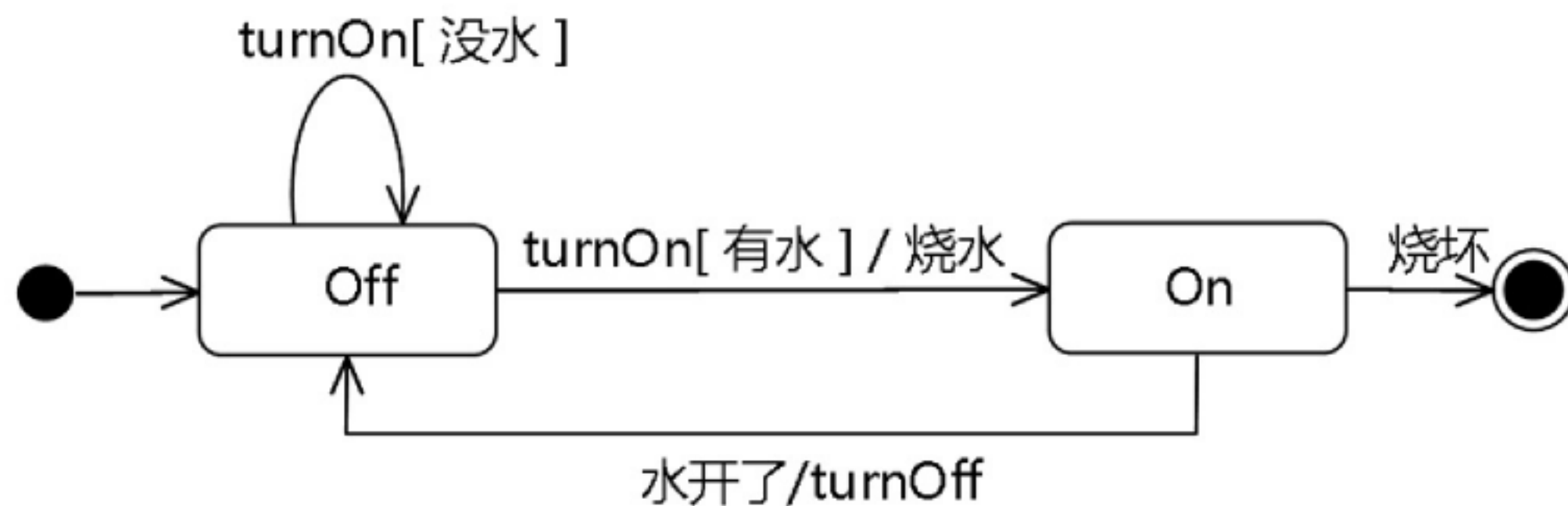




状态图



状态图（state diagram）是对类描述的补充。用于展现此类对象所具有的可能状态，以及某些事件发生时其状态转移情况。





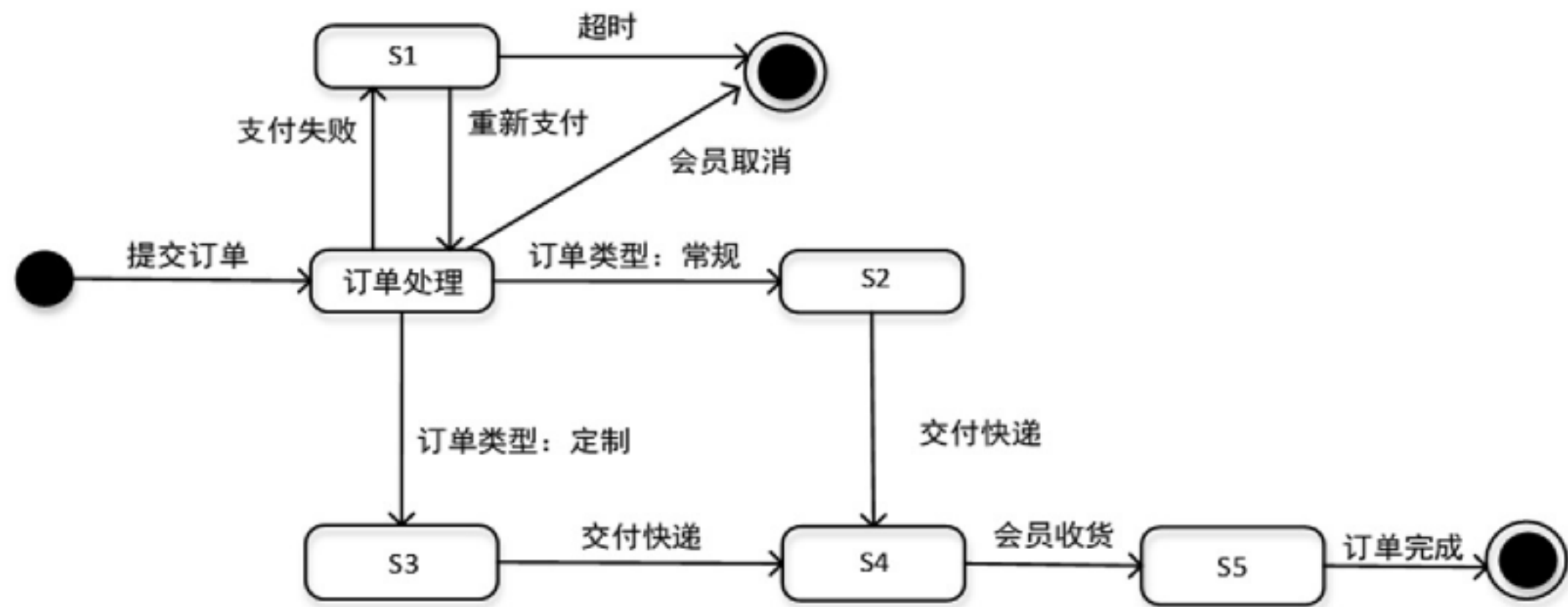
状态图



在订单处理的过程中，会员可以点击“取消订单”按钮取消该订单。如果**支付失败**，该订单将被标记为**挂起状态**，可后续重新支付，如果挂起超时30分钟未支付，系统将自动取消该订单。订单支付成功后，系统判断订单类型：

(1) 对于常规订单，标记为**备货状态**，订单信息发送到货运部，完成打包后**交付快递**发货；

(2) 对于定制订单，会自动进入**定制状态**，定制完成后**交付快递**发货。会员在系统中点击“**收货**”按钮变为**收货状态**，结束整个订单的处理流程。





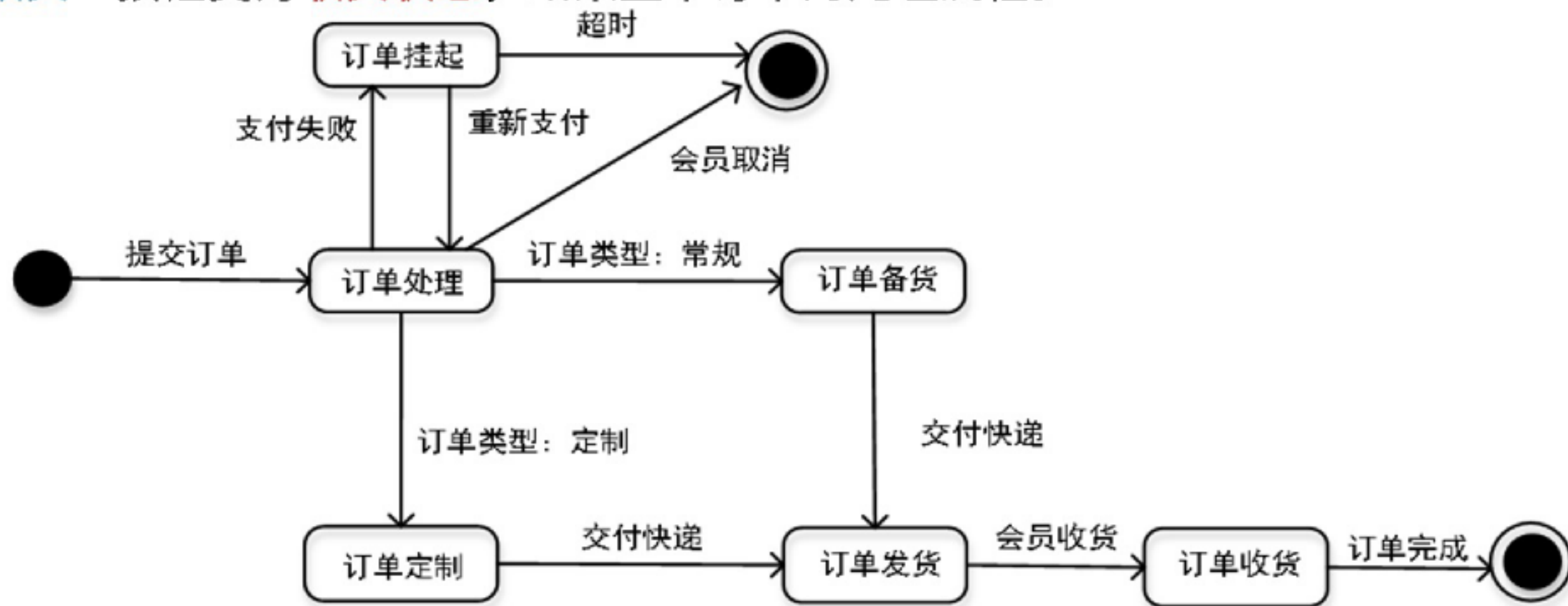
状态图



在订单处理的过程中，会员可以点击“取消订单”按钮取消该订单。如果**支付失败**，该订单将被标记为**挂起状态**，可后续重新支付，如果挂起超时30分钟未支付，系统将自动取消该订单。

订单支付成功后，系统判断订单类型：

- (1) 对于常规订单，标记为**备货状态**，订单信息发送到货运部，完成打包后**交付快递**发货；
- (2) 对于定制订单，会自动进入**定制状态**，定制完成后**交付快递**发货。会员在系统中点击“**收货**”按钮变为**收货状态**，结束整个订单的处理流程。



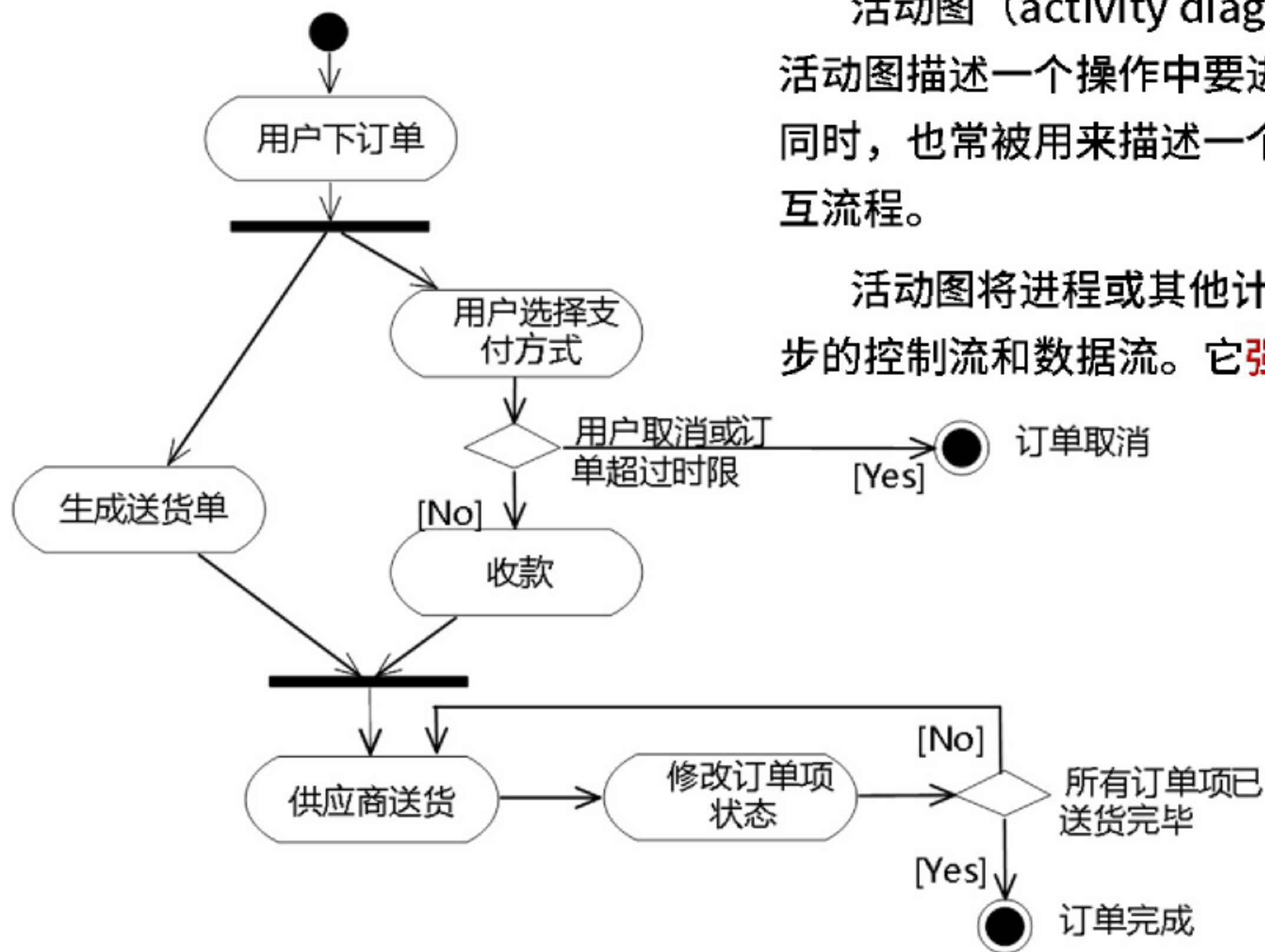


活动图



活动图（activity diagram）是一种特殊的状态图。活动图描述一个操作中要进行的各项活动的执行流程。同时，也常被用来描述一个用例的处理流程或者某种交互流程。

活动图将进程或其他计算结构展示为计算内部一步步的控制流和数据流。它**强调对象间的控制流程**。



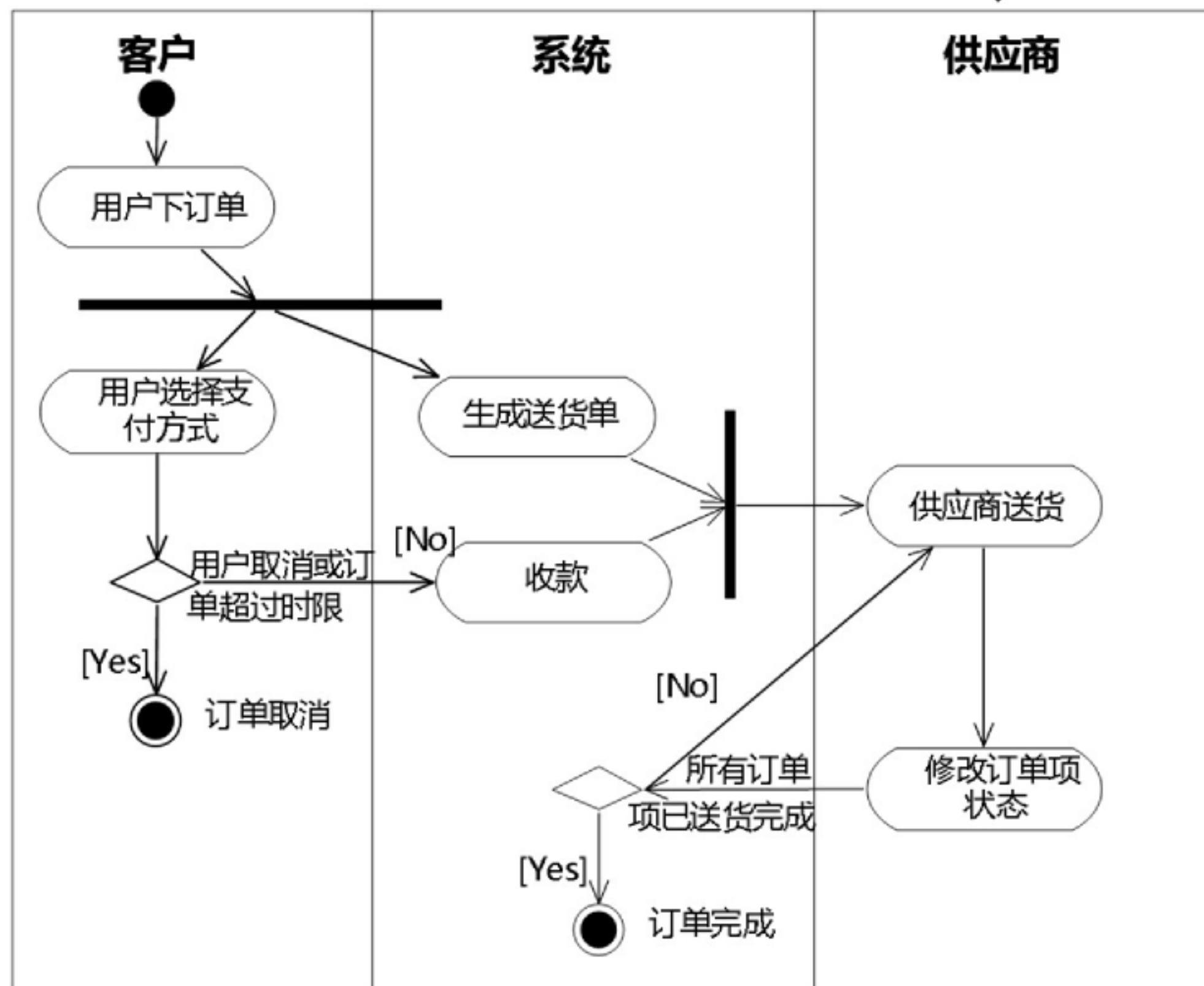


泳道式活动图

泳道



希赛





习题讲解



希赛公司欲开发一个在线交易系统。为了能够精确表达用户与系统的复杂交互过程，应该采用UML的（ ）进行交互过程建模。

- A. 类图 B. 顺序图 C. 部署图 D. 对象图

（ ）适用于描述复杂算法的执行流程。

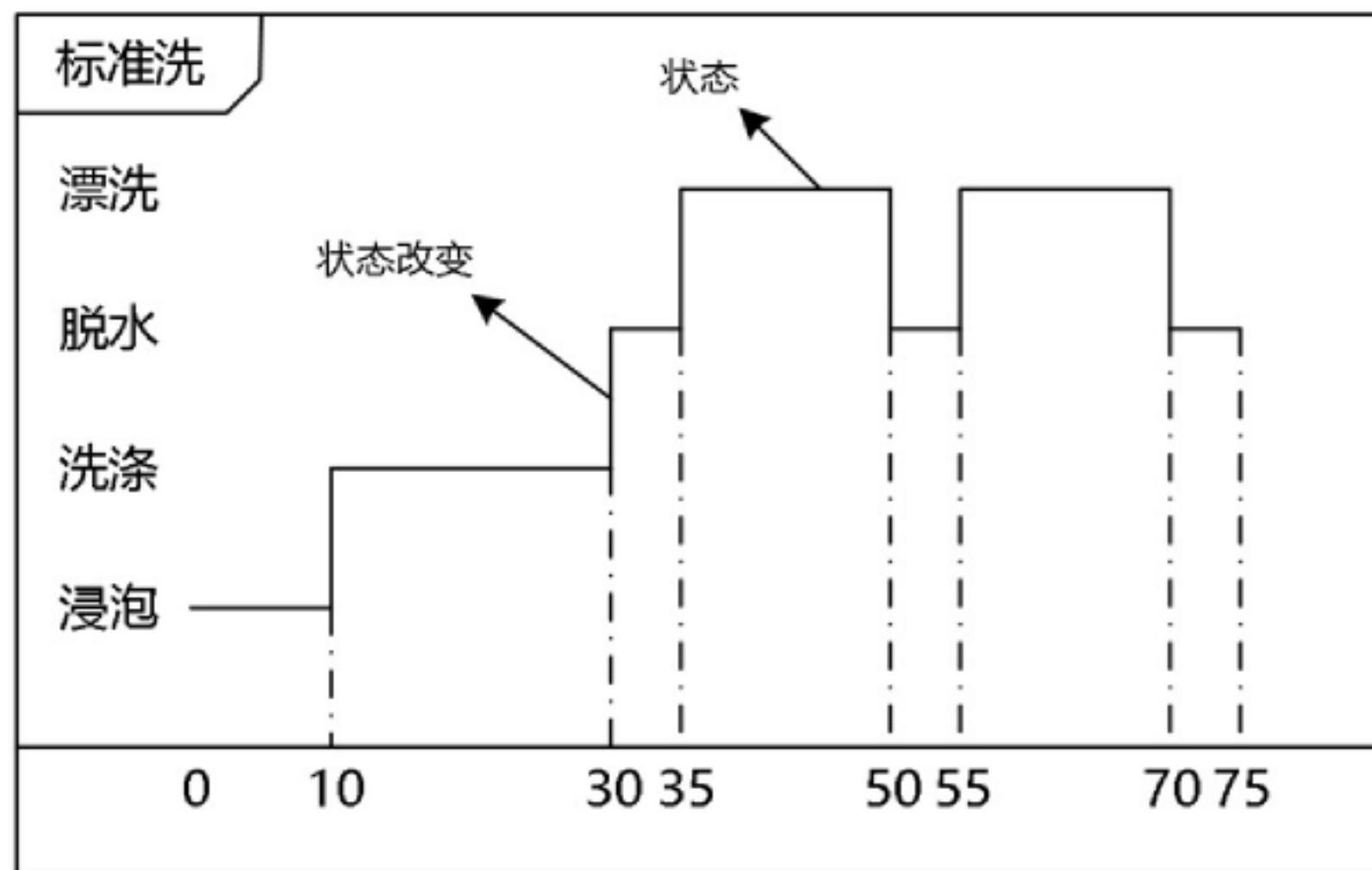
- A. 活动图 B. 状态图 C. 类图 D. 用例图



定时图



定时图也叫计时图，也是一种交互图，用于展示交互过程中的真实时间信息，具体描述对象状态变化的时间点以及维持特定状态的时间段。

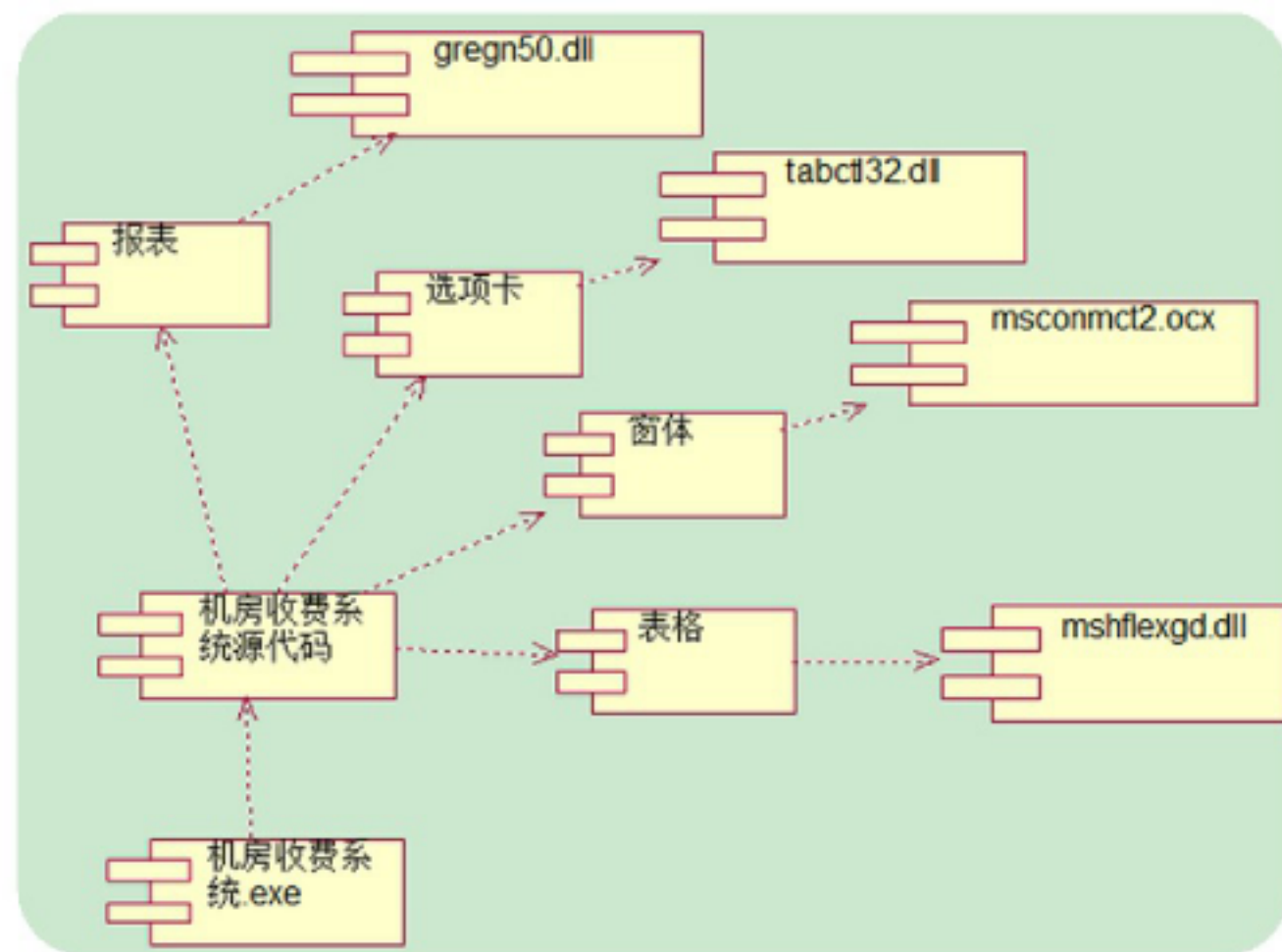




构件图与包图



构件图（component diagram）。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。对于由小的部件构建大的系统来说，构件图是很重要的。构件图是类图的变体。



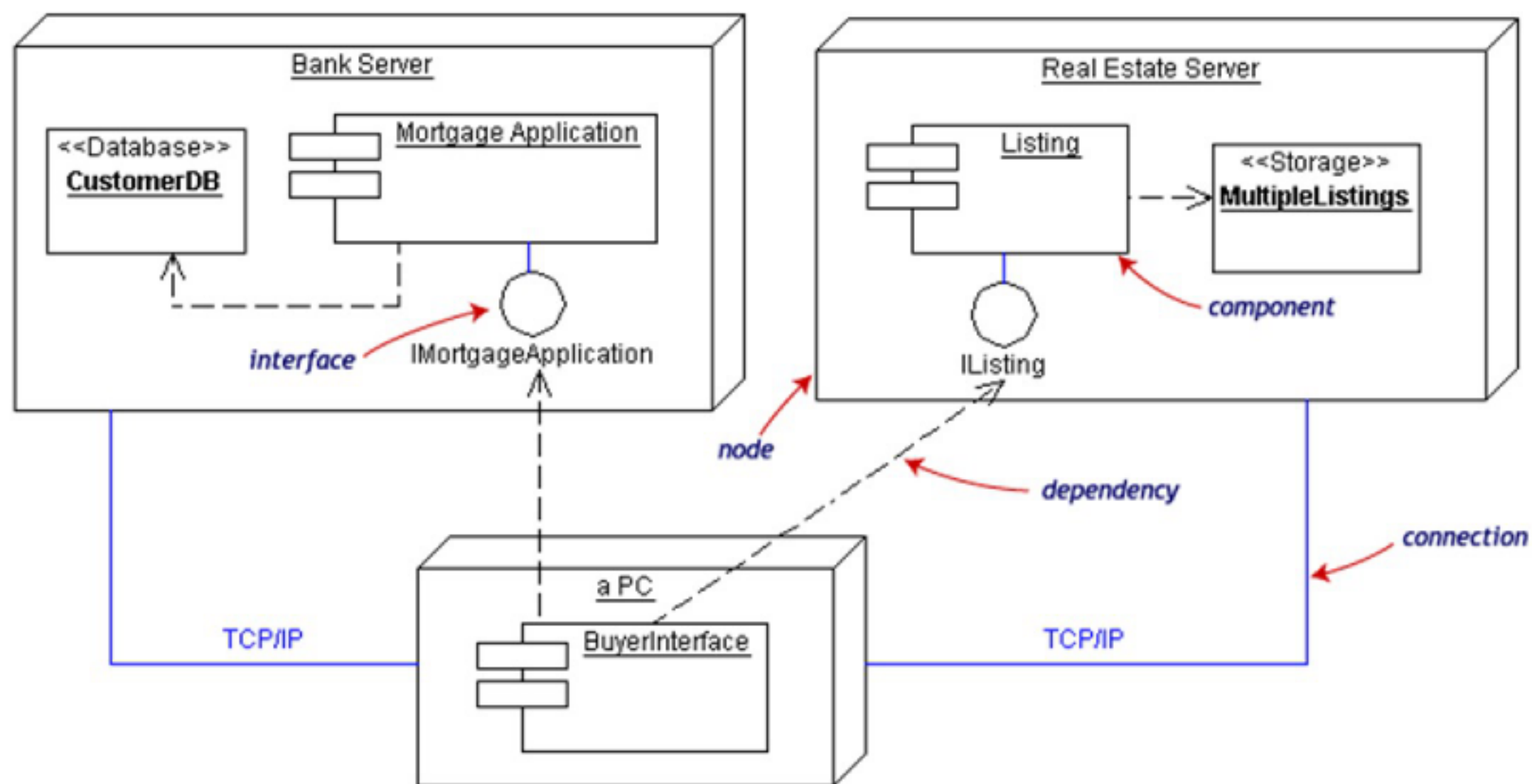
包图，包的图标像是一个带标签的文件夹，包的基本思想是把共同工作的元素放到一个文件夹中。例：多个类或构件组成了一个子系统，就可以将它们放到一个包中。



部署图



部署图（deployment diagram）。部署图描述对运行时的处理节点及在其中生存的构件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。





案例分析练习题



某软件公司拟为物流企业开发一套库存管理系统，该系统的部分需求陈述如下：

- (1) 库存管理系统主要包括货物入库管理、货物出库管理、仓库管理、统计报表和系统管理等功能。
- (2) 库存管理系统的用户包括仓库管理员、仓库经理和系统管理员，用户必须在注册后才能使用系统功能；用户可以选择使用邮件注册或电话注册。
- (3) 仓库管理员在进行出入库操作前必须先登录；仓库经理可以通过系统查看统计报表，如果前一个月的报表未生成，则系统自动生成统计报表，否则直接显示。
- (4) 系统管理员可以在系统中设置仓库温度范围，当仓库内温度超过最高值或者低于最低值时，系统自动调用温控管理操作，连接温度调节系统进行制冷或加热。
- (5) 仓库管理功能要求每个月1日零点对前一个月货物入库和出库记录进行数据汇总操作。项目组决定构造用例模型以描述系统需求。

【问题1】（6分）

用例建模的首要任务是识别系统中的参与者。请根据题目中所描述的需求，识别出系统中有哪些参与者？



案例分析练习题

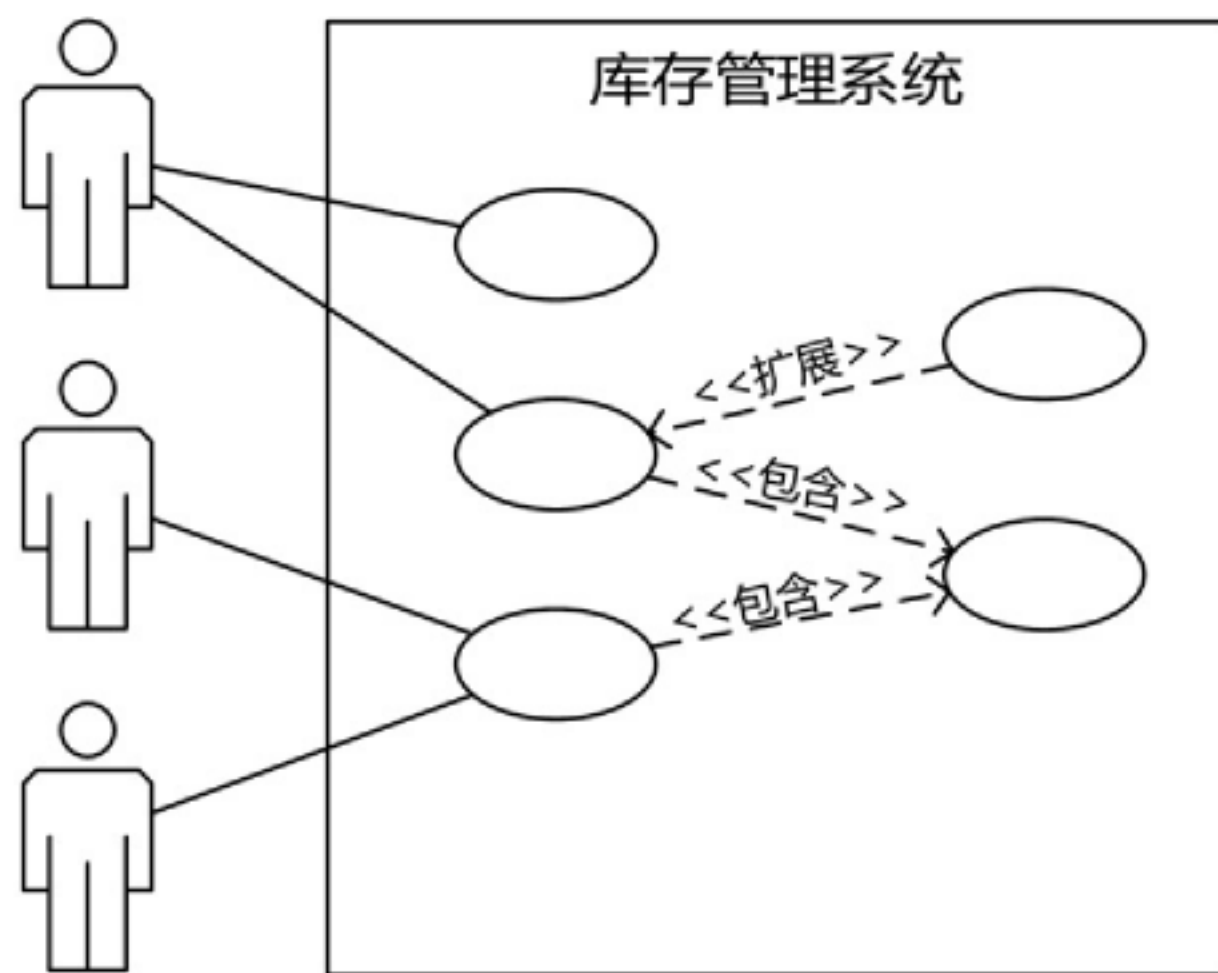


【问题2】（7分）

用例建模的主要工作是书写用例规约。用例规约通常包括哪几部分内容？

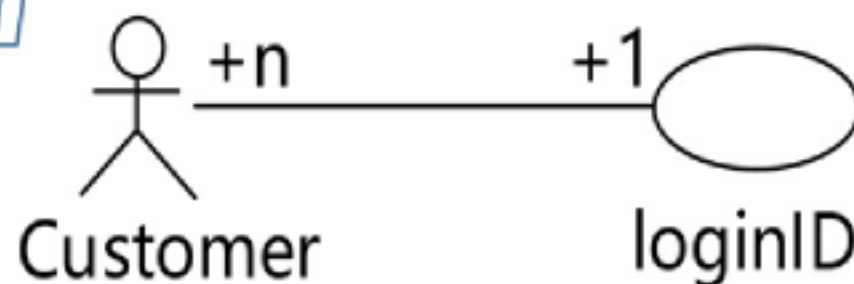
【问题3】（12分）

建立了用例模型后，可以利用用例之间的关系调整用例模型，用例之间的关系包括哪几种？对于每种关系，请根据题目中所描述的需求分别给出一组用例。





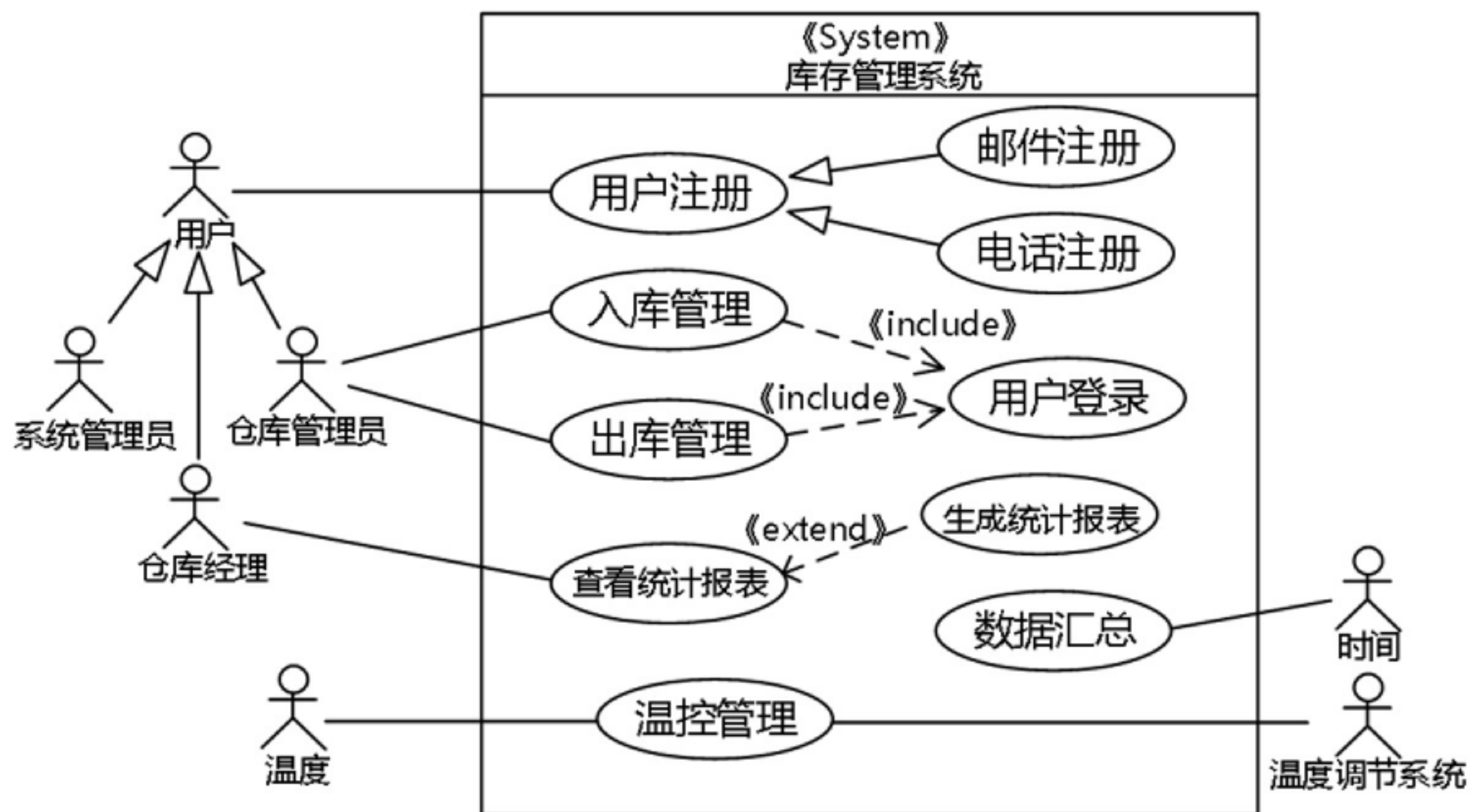
案例分析练习题



用例名称:	登录	
用例ID:	EDU_SHOP_002.1	
角色:	普通用户	
用例说明:	用例主要功能是实现登录, 起始于普通用户的登录	
前置条件:	启动程序, 进入登录界面	
基本事件流:	参与者动作	系统响应
	1.用户输入基本信息(登录名和密码), 点击确定按钮	2.系统查找数据库, 看该用户是否在数据库中。若存在则进入主页面; 若不存在, 则进入2.2.2; 若未输入, 则进入2.2.2
其他事件流:	无	
异常事件流:	参与者动作	系统响应
	2.2.2未输入用户名 2.2.1用户名不存在 2.1.2未输入密码 2.2.2密码不正确	2.1.1提示用户名或密码不能为空 2.2.2提示用户名或密码不正确
后置条件	登录成功	



案例分析练习题





案例分析练习题- 参考答案



【问题1】

用例模型的参与者：仓库管理员、仓库经理、系统管理员、时间、温度、温度调节系统。

【问题2】

用例名称、简要说明、事件流、非功能需求、前置条件、后置条件、扩展点、优先级。

【问题3】

(1) 用例之间的关系包括：包含关系、扩展关系、泛化关系。

(2) “出入库操作”与“登录”属于包含关系；“查看统计报表”与“生成统计报表”属于扩展关系；“用户注册”与“邮件注册”和“电话注册”属于典型的泛化关系。



案例分析练习题



某软件公司为电子商务企业开发一套网上交易订单管理系统，以提升服务的质量和效率。在项目之初，项目组决定采用面向对象的开发方法进行系统开发，并对系统的核心业务功能进行了分析，具体描述如下：

注册用户通过商品信息页面在线浏览商品，将需要购买的商品添加进购物车内，点击“结算”按钮后开始录入订单信息。

用户在订单信息录入页面上选择支付方式，填写并确认收货人、收货地址和联系方式等信息。点击“提交订单”按钮后产生订单，并开始进行订单结算。

订单需要在30分钟内进行支付，否则会自动取消，用户也可以手工取消订单。

用户支付完成，经确认后，系统开始备货，扣除该商品可接单数量，并移除用户购物车中的所有商品资料。

生成订单表单，出货完毕，订单生效。为用户快递商品，等待用户接收。

用户签收商品，交易完成。

【问题1】（12分）

识别设计类是面向对象设计过程中的重要工作，设计类表达了类的职责，即该类所担任的任务。请用300字以内的文字说明设计类通常分为哪三种类型，每种类型的主要职责，并针对题干描述案例涉及的具体类为每种类型的设计类举出2个实例。

【问题2】（3分）

在面向对象的设计过程中，活动图（activity diagram）阐明了业务用例实现的工作流程。请用300字以内的文字给出活动图与流程图（flow chart）的三个主要区别。

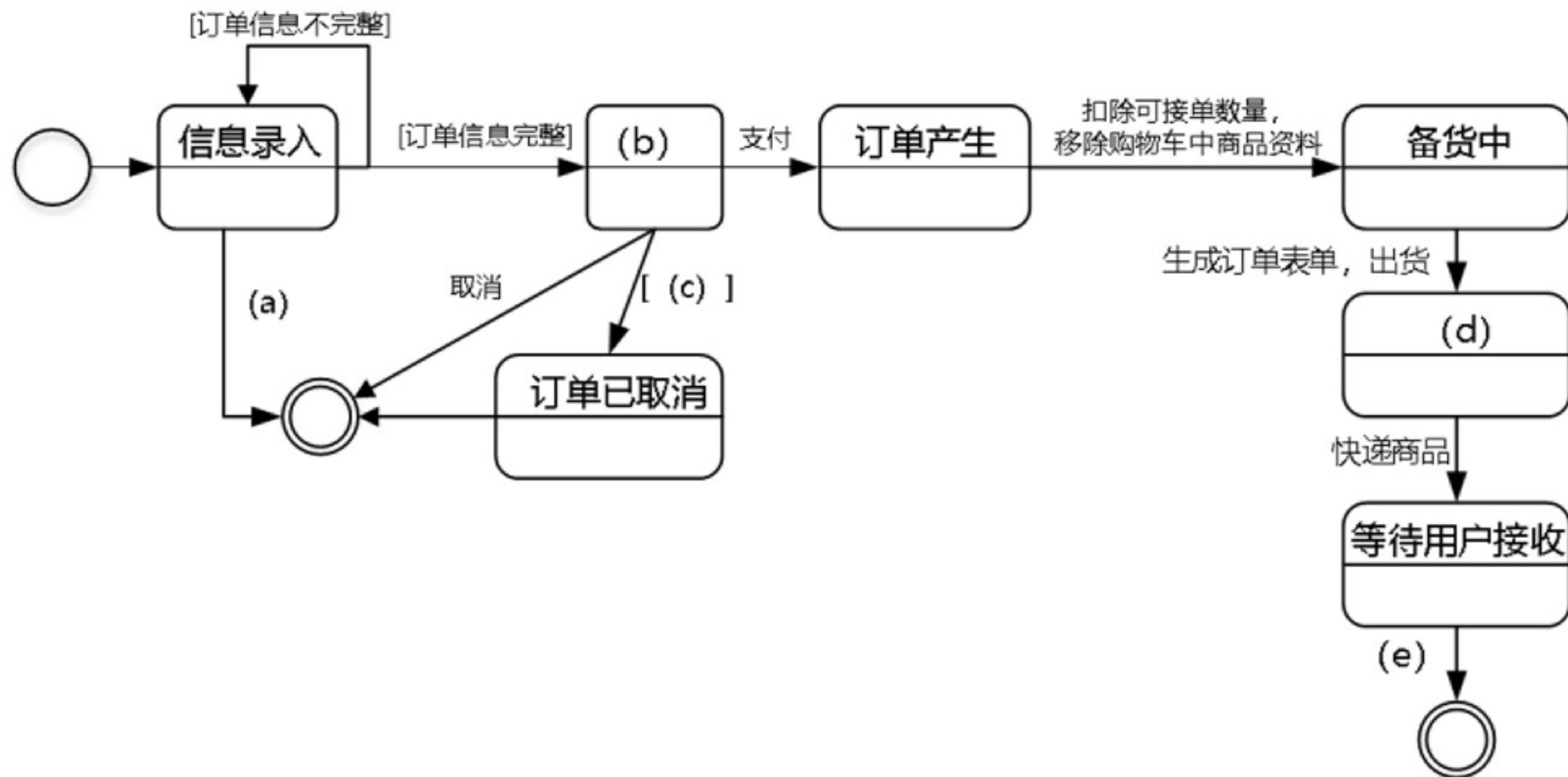


案例分析练习题



【问题3】（10分）

在面向对象的设计过程中，状态图（statechart diagram）描述了一个实体基于事件反应的动态行为。请根据题干描述，填写下图中的（a）~（e）空白，完成订单处理的状态图。





案例分析练习题- 参考答案



【问题1】

- (1) 实体类。实体类映射需求中的每个实体，保存需要存储在永久存储体中的信息，例如，用户、商品等。
- (2) 控制类。控制类是用于控制用例工作的类，用于对一个或几个用例所特有的控制行为进行建模。例如，结算、备货等。
- (3) 边界类。边界类用于封装在用例内、外流动的信息或数据流。例如，浏览器、购物车等。

【问题2】

- (1) 活动图描述的是对象活动的顺序关系所遵循的规则，它着重表现系统的行为，而非处理过程；而流程图着重描述处理过程。
- (2) 流程图一般都限于顺序进程，而活动图则可以支持并发进程。
- (3) 活动图是面向对象的，而流程图是面向过程的。

【问题3】

- (a) 取消 (b) 待结算 (c) 大于30分钟 (d) 订单生效 (e) 用户签收



系统工程建模



MBSE (Model Based Systems Engineering, 基于模型的**系统工程**)

【MBSE的三大支柱】

【软件工程】

【建模语言】 沟通标准化，MBSE的核心。典型代表：**SysML**，OPM。-----> **UML**

【建模方法】 类似于路线图，如：INCONSE面向对象系统工程法。-----> **面向对象开发**

【建模工具】 如：Agilian，Modelio。-----> **Visio、Rose**

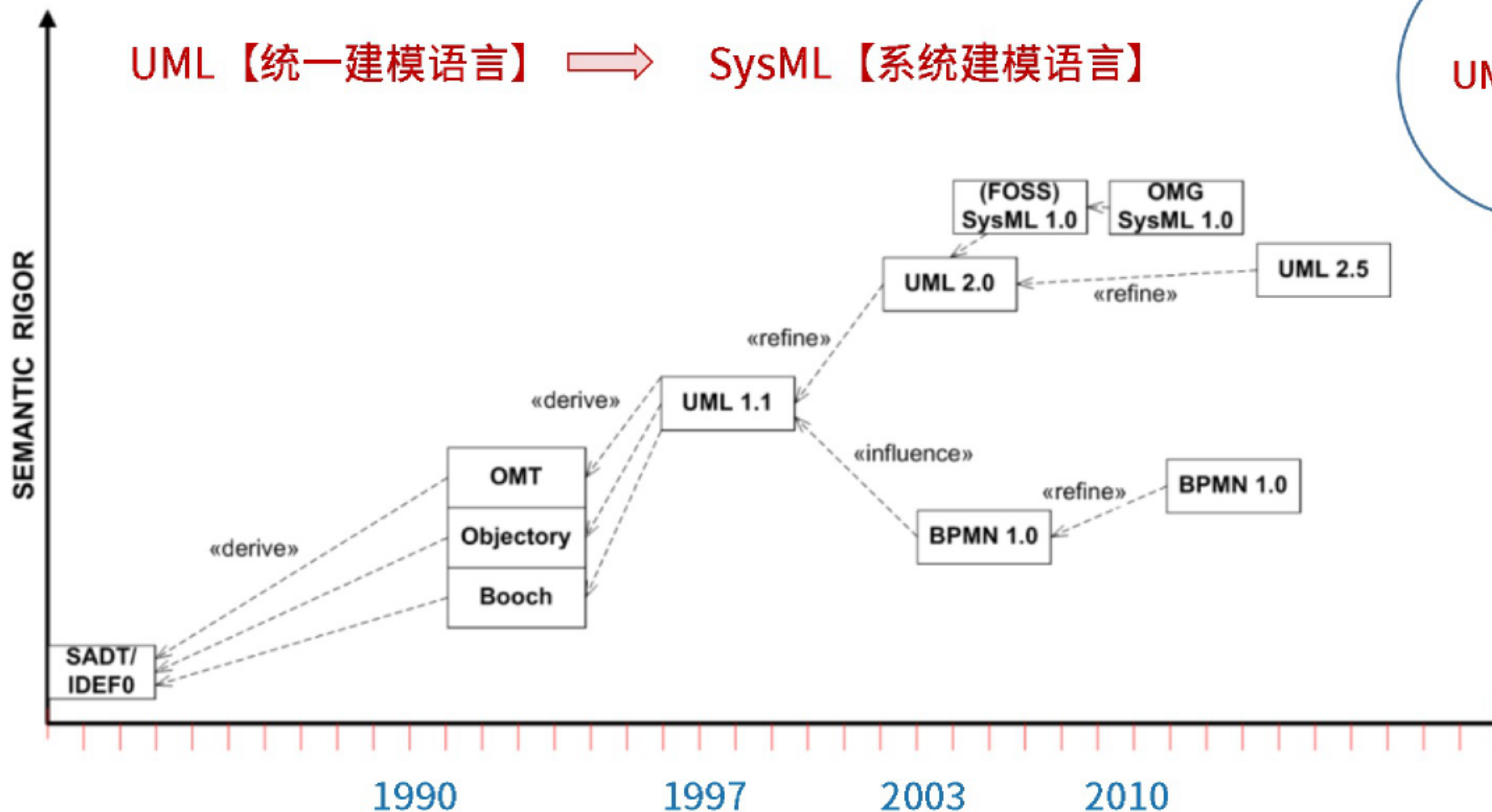
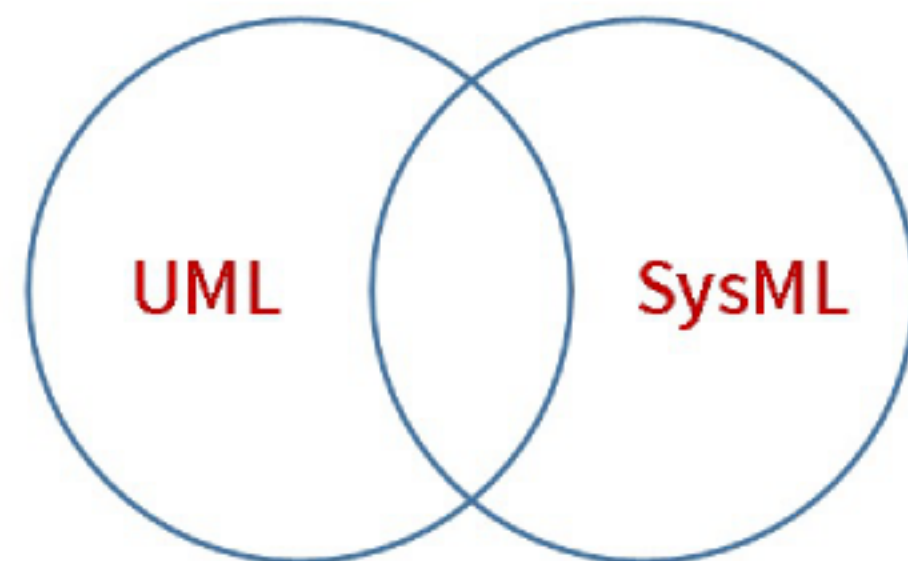




系统工程建模 – SysML

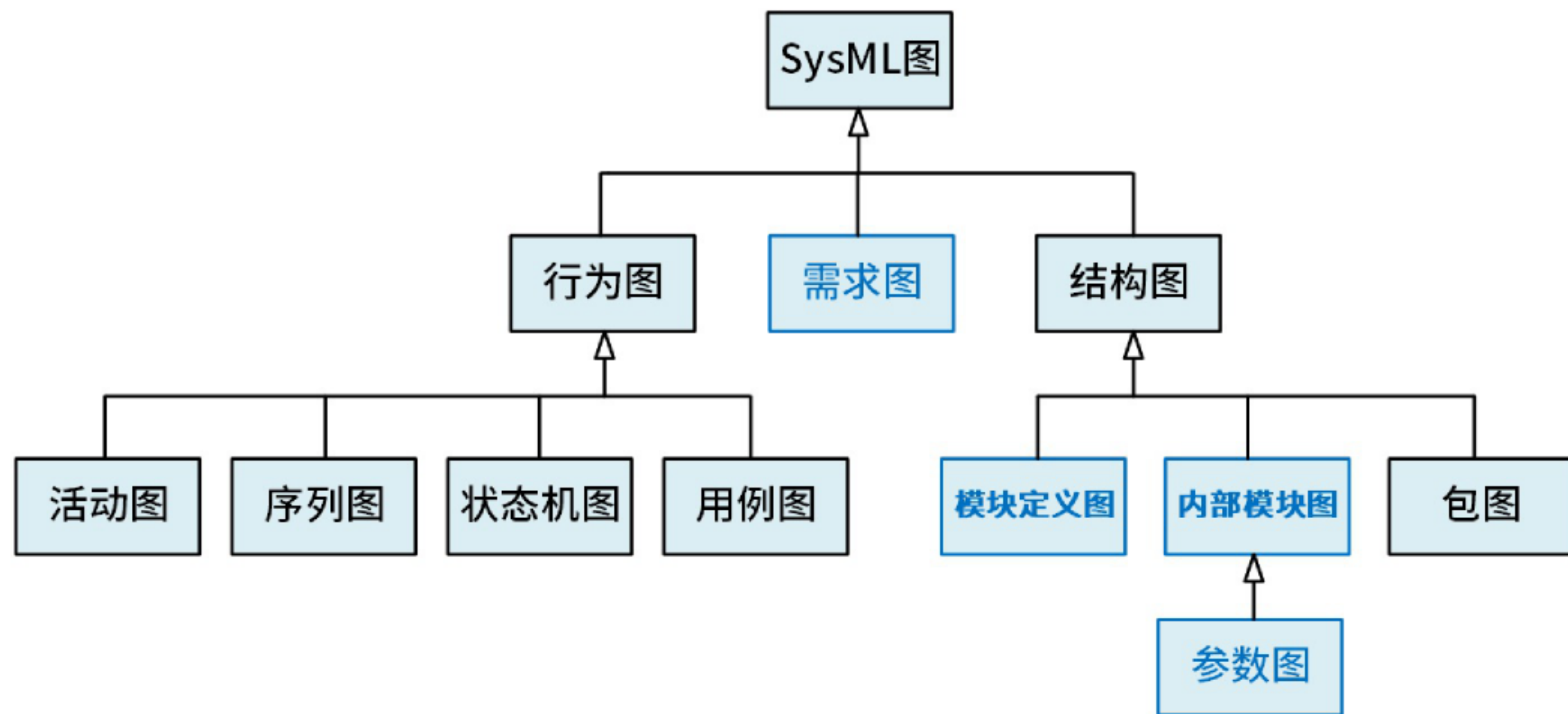


UML 【统一建模语言】 → SysML 【系统建模语言】



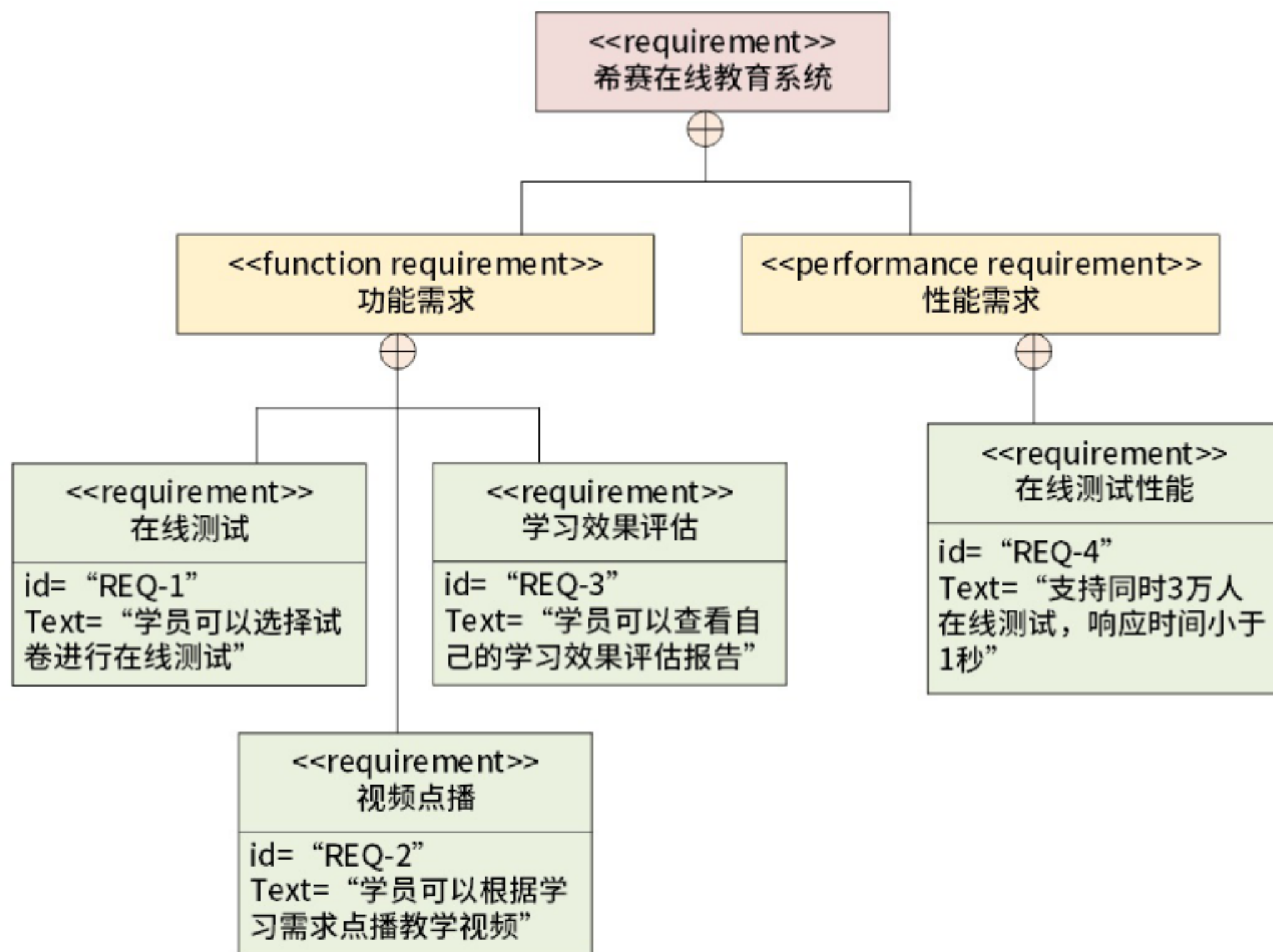


系统工程建模 – SysML



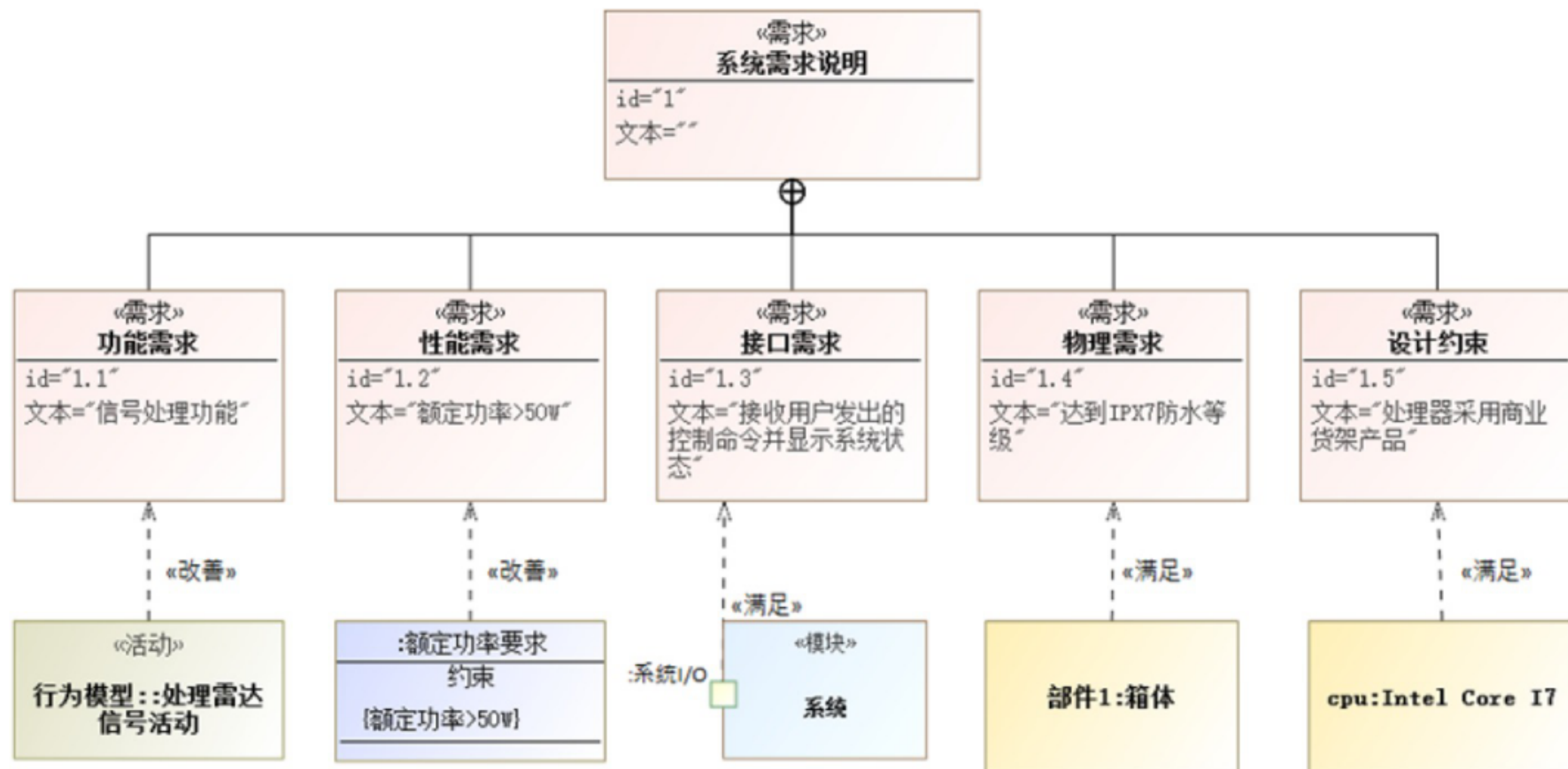


系统工程建模 – SysML 【需求图】





系统工程建模 – SysML 【需求图】





系统工程建模 – SysML 【需求图】



SysML中的需求关系：**包含、跟踪、继承需求、改善、满足、验证和复制。**

包含 (Include)：需求可以且只能包含其他需求。

跟踪 (Trace)：对提供方元素（位于箭头端）的修改可能会导致对客户端元素（位于尾端）修改的需要。

继承 (deriveReq)：一个需求可以继承另一个需求的属性。

改善 (refine)：表示一个需求改进了另一个需求的满足程度。

满足 (satisfy)：一般是模块满足某种需求。

验证 (verify)：表示一个需求验证了另一个需求的正确性。

复制 (Copy)：表示一个需求复制了另一个需求的特性。

【依赖关系】



系统工程建模 – SysML 【需求图】



SysML中的需求关系：包含、跟踪、继承需求、改善、满足、验证和复制。

